1.1 Introduction

Internet of Things (IoT) comprises things that have unique identities and are connected to the Internet. While many existing devices, such as networked computers or 4G-enabled mobile phones, already have some form of unique identities and are also connected to the Internet, the focus on IoT is in the configuration, control and networking via the Internet of devices or "things" that are traditionally not associated with the Internet. These include devices such as thermostats, utility meters, a bluetooth-connected headset, irrigation pumps and sensors, or control circuits for an electric car's engine. Internet of Things is a new revolution in the capabilities of the endpoints that are connected to the Internet, and is being driven by the advancements in capabilities (in combination with lower costs) in sensor networks, mobile devices, wireless communications, networking and cloud technologies. Experts forecast that by the year 2020 there will be a total of 50 billion devices/things connected to the Internet. Therefore, the major industry players are excited by the prospects of new markets for their products. The products include hardware and software components for IoT endpoints, hubs, or control centers of the IoT universe.

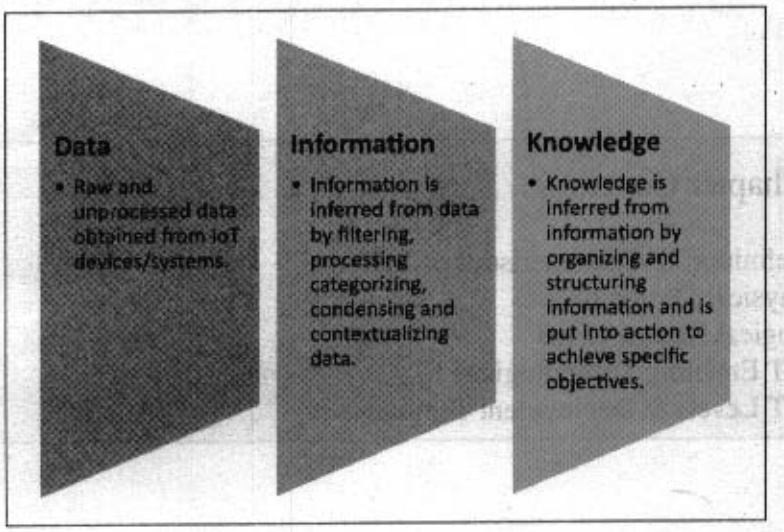


Figure 1.1: Inferring information and knowledge from data

The scope of IoT is not limited to just connecting things (devices, appliances, machines) to the Internet. IoT allows these things to communicate and exchange data (control & information, that could include data associated with users) while executing meaningful applications towards a common user or machine goal. Data itself does not have a meaning

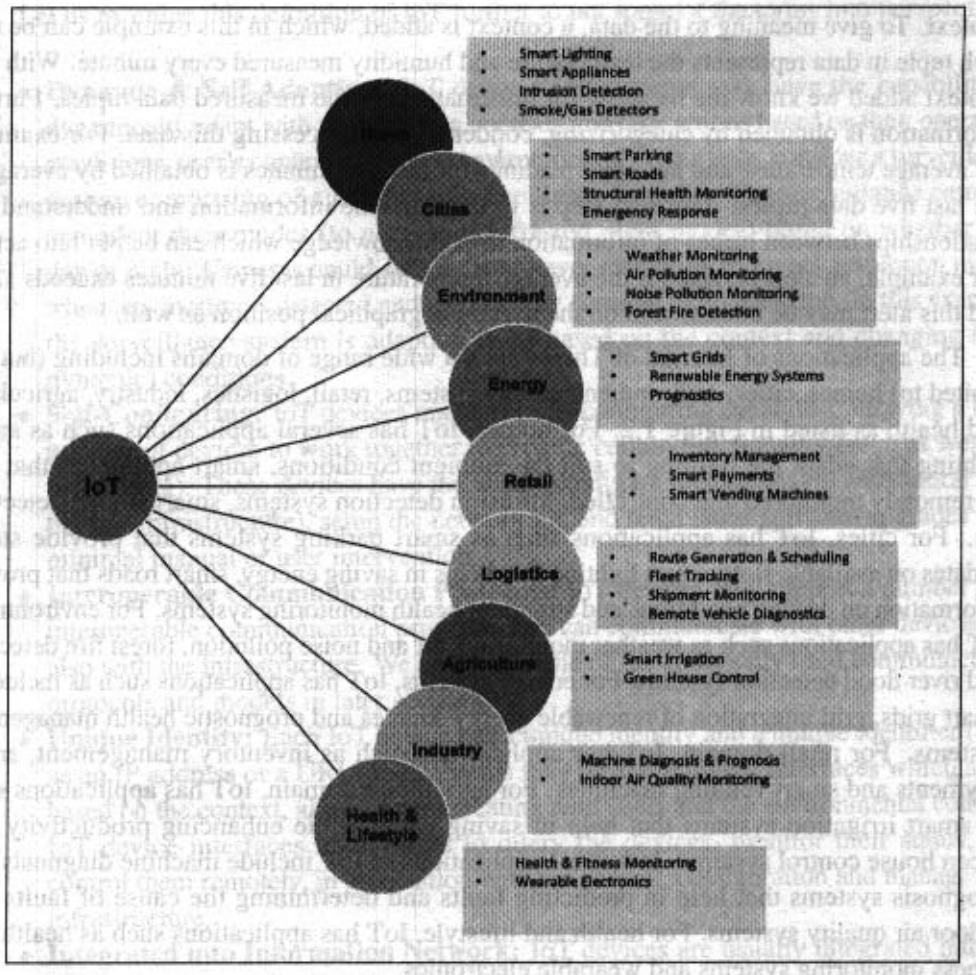


Figure 1.2: Applications of IoT

until it is contextualized processed into useful information. Applications on IoT networks extract and create information from lower level data by filtering, processing, categorizing, condensing and contextualizing the data. This information obtained is then organized and structured to infer knowledge about the system and/or its users, its environment, and its operations and progress towards its objectives, allowing a smarter performance, as shown in Figure 1.1. For example, consider a series of raw sensor measurements ((72,45) ; (84, 56)) generated by a weather monitoring station, which by themselves do not have any meaning or

context. To give meaning to the data, a context is added, which in this example can be that each tuple in data represents the temperature and humidity measured every minute. With this context added we know the meaning (or information) of the measured data tuples. Further information is obtained by categorizing, condensing or processing this data. For example, the average temperature and humidity readings for last five minutes is obtained by averaging the last five data tuples. The next step is to organize the information and understand the relationships between pieces of information to infer knowledge which can be put into action. For example, an alert is raised if the average temperature in last five minutes exceeds 120F, and this alert may be conditioned on the user's geographical position as well.

The applications of Internet of Things span a wide range of domains including (but not limited to) homes, cities, environment, energy systems, retail, logistics, industry, agriculture and health as listed in Figure 1.2. For homes, IoT has several applications such as smart lighting that adapt the lighting to suit the ambient conditions, smart appliances that can be remotely monitored and controlled, intrusion detection systems, smart smoke detectors, etc. For cities, IoT has applications such as smart parking systems that provide status updates on available slots, smart lighting that helps in saving energy, smart roads that provide information on driving conditions and structural health monitoring systems. For environment, IoT has applications such as weather monitoring, air and noise pollution, forest fire detection and river flood detection systems. For energy systems, IoT has applications such as including smart grids, grid integration of renewable energy sources and prognostic health management systems. For retail domain, IoT has applications such as inventory management, smart payments and smart vending machines. For agriculture domain, IoT has applications such as smart irrigation systems that help in saving water while enhancing productivity and green house control systems. Industrial applications of IoT include machine diagnosis and prognosis systems that help in predicting faults and determining the cause of faults and indoor air quality systems. For health and lifestyle, IoT has applications such as health and fitness monitoring systems and wearable electronics.

1.1.1 Definition & Characteristics of IoT

The Internet of Things (IoT) has been defined as [1]:

> **Definition:** A dynamic global network infrastructure with self-configuring capabilities based on standard and interoperable communication protocols where physical and virtual "things" have identities, physical attributes, and virtual personalities and use intelligent interfaces, and are seamlessly integrated into the information network, often communicate data associated with users and their environments.

Let us examine this definition of IoT further to put some of the terms into perspective.

- **Dynamic & Self-Adapting:** IoT devices and systems may have the capability to dynamically adapt with the changing contexts and take actions based on their operating conditions, user's context, or sensed environment. For example, consider a surveillance system comprising of a number of surveillance cameras. The surveillance cameras can adapt their modes (to normal or infra-red night modes) based on whether it is day or night. Cameras could switch from lower resolution to higher resolution modes when any motion is detected and alert nearby cameras to do the same. In this example, the surveillance system is adapting itself based on the context and changing (e.g., dynamic) conditions.
- **Self-Configuring:** IoT devices may have self-configuring capability, allowing a large number of devices to work together to provide certain functionality (such as weather monitoring). These devices have the ability configure themselves (in association with the IoT infrastructure), setup the networking, and fetch latest software upgrades with minimal manual or user intervention.
- **Interoperable Communication Protocols:** IoT devices may support a number of interoperable communication protocols and can communicate with other devices and also with the infrastructure. We describe some of the commonly used communication protocols and models in later sections.
- **Unique Identity:** Each IoT device has a unique identity and a unique identifier (such as an IP address or a URI). IoT systems may have intelligent interfaces which adapt based on the context, allow communicating with users and the environmental contexts. IoT device interfaces allow users to query the devices, monitor their status, and control them remotely, in association with the control, configuration and management infrastructure.
- **Integrated into Information Network:** IoT devices are usually integrated into the information network that allows them to communicate and exchange data with other devices and systems. IoT devices can be dynamically discovered in the network, by other devices and/or the network, and have the capability to describe themselves (and their characteristics) to other devices or user applications. For example, a weather monitoring node can describe its monitoring capabilities to another connected node so that they can communicate and exchange data. Integration into the information network helps in making IoT systems "smarter" due to the collective intelligence of the individual devices in collaboration with the infrastructure. Thus, the data from a large number of connected weather monitoring IoT nodes can be aggregated and analyzed to predict the weather.

1.2 Physical Design of IoT

1.2.1 Things in IoT

The "Things" in IoT usually refers to IoT devices which have unique identities and can perform remote sensing, actuating and monitoring capabilities. IoT devices can exchange data with other connected devices and applications (directly or indirectly), or collect data from other devices and process the data either locally or send the data to centralized servers or cloud-based application back-ends for processing the data, or perform some tasks locally and other tasks within the IoT infrastructure, based on temporal and space constraints (i.e., memory, processing capabilities, communication latencies and speeds, and deadlines).

Figure 1.3 shows a block diagram of a typical IoT device. An IoT device may consist of several interfaces for connections to other devices, both wired and wireless. These include (i) I/O interfaces for sensors, (ii) interfaces for Internet connectivity, (iii) memory and storage interfaces and (iv) audio/video interfaces. An IoT device can collect various types of data from the on-board or attached sensors, such as temperature, humidity, light intensity. The sensed data can be communicated either to other devices or cloud-based servers/storage. IoT devices can be connected to actuators that allow them to interact with other physical entities (including non-IoT devices and systems) in the vicinity of the device. For example, a relay switch connected to an IoT device can turn an appliance on/off based on the commands sent to the IoT device over the Internet.

IoT devices can also be of varied types, for instance, wearable sensors, smart watches, LED lights, automobiles and industrial machines. Almost all IoT devices generate data in some form or the other which when processed by data analytics systems leads to useful information to guide further actions locally or remotely. For instance, sensor data generated by a soil moisture monitoring device in a garden, when processed can help in determining the optimum watering schedules. Figure 1.4 shows different types of IoT devices.

1.2.2 IoT Protocols

Link Layer

Link layer protocols determine how the data is physically sent over the network's physical layer or medium (e.g., copper wire, coaxial cable, or a radio wave). The scope of the link layer is the local network connection to which host is attached. Hosts on the same link exchange data packets over the link layer using link layer protocols. Link layer determines how the packets are coded and signaled by the hardware device over the medium to which the host is attached (such as a coaxial cable). Let us now look at some link layer protocols which are relevant in the context of IoT.

- **802.3 - Ethernet :** IEEE 802.3 is a collection of wired Ethernet standards for the link

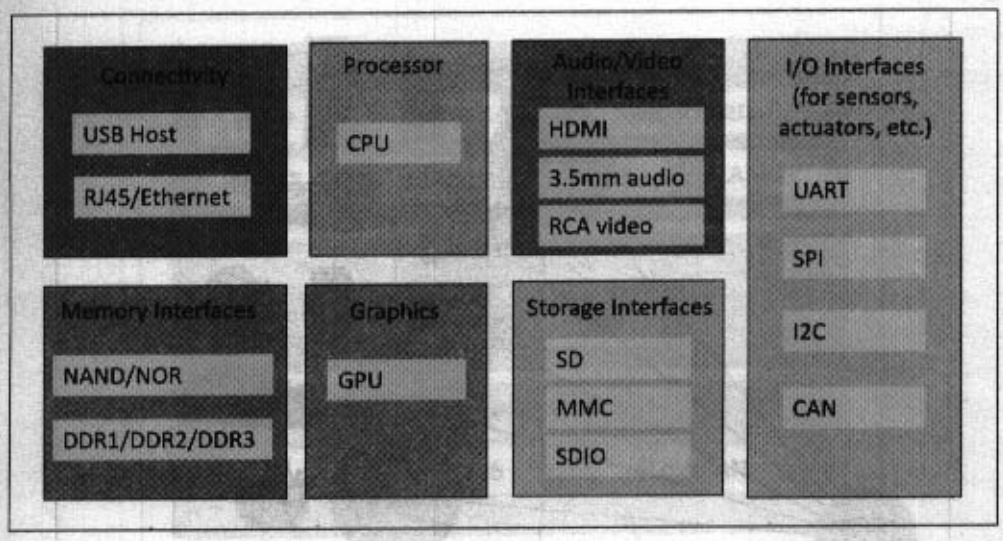


Figure 1.3: Generic block diagram of an IoT Device

layer. For example, 802.3 is the standard for 10BASE5 Ethernet that uses coaxial cable as a shared medium, 802.3.i is the standard for 10BASE-T Ethernet over copper twisted-pair connections, 802.3.j is the standard for 10BASE-F Ethernet over fiber optic connections, 802.3ae is the standard for 10 Gbit/s Ethernet over fiber, and so on. These standards provide data rates from 10 Mb/s to 40 Gb/s and higher. The shared medium in Ethernet can be a coaxial cable, twisted-pair wire or an optical fiber. The shared medium (i.e., broadcast medium) carries the communication for all the devices on the network, thus data sent by one device can received by all devices subject to propagation conditions and transceiver capabilities. The specifications of the 802.3 standards are available on the IEEE 802.3 working group website [2].

- **802.11 - WiFi :** IEEE 802.11 is a collection of wireless local area network (WLAN) communication standards, including extensive description of the link layer. For example, 802.11a operates in the 5 GHz band, 802.11b and 802.11g operate in the 2.4 GHz band, 802.11n operates in the 2.4/5 GHz bands, 802.11ac operates in the 5 GHz band and 802.11ad operates in the 60 GHz band. These standards provide data rates from 1 Mb/s to upto 6.75 Gb/s. The specifications of the 802.11 standards are available on the IEEE 802.11 working group website [3]
- **802.16 - WiMax :** IEEE 802.16 is a collection of wireless broadband standards, including extensive descriptions for the link layer (also called WiMax). WiMax

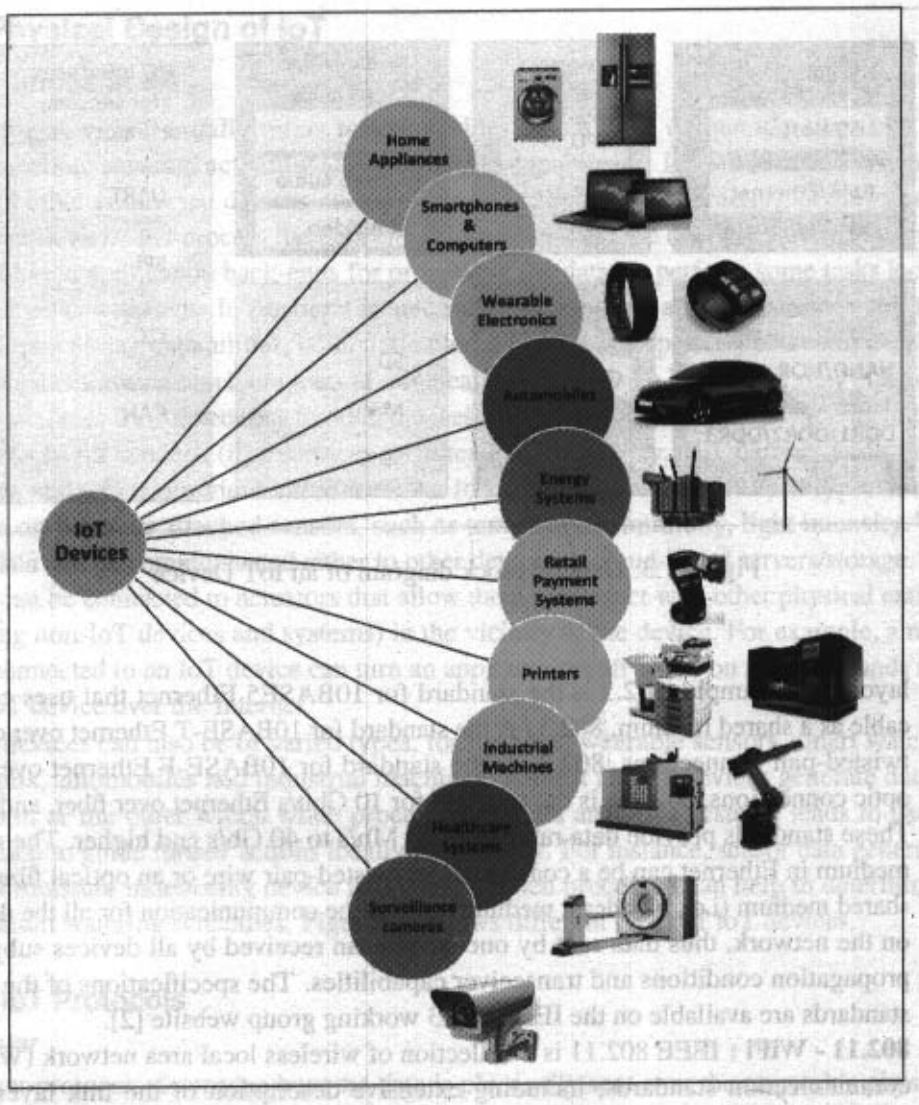


Figure 1.4: IoT Devices

standards provide data rates from 1.5 Mb/s to 1 Gb/s. The recent update (802.16m) provides data rates of 100 Mbit/s for mobile stations and 1 Gbit/s for fixed stations. The specifications of the 802.11 standards are readily available on the IEEE 802.16 working group website [4]

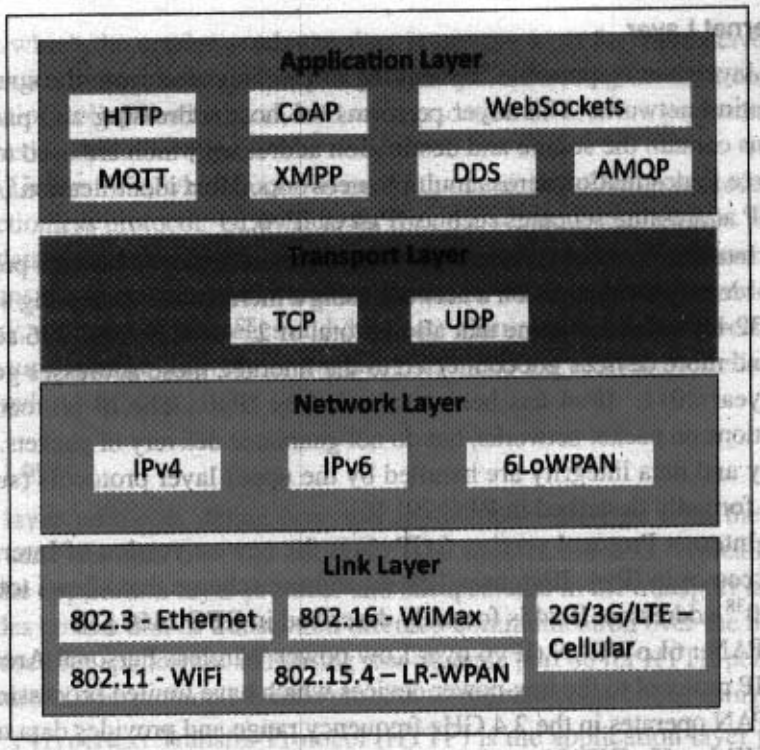


Figure 1.5: IoT Protocols

- **802.15.4 - LR-WPAN :** IEEE 802.15.4 is a collection of standards for low-rate wireless personal area networks (LR-WPANs). These standards form the basis of specifications for high level communication protocols such as ZigBee. LR-WPAN standards provide data rates from 40 Kb/s 250 Kb/s. These standards provide low-cost and low-speed communication for power constrained devices. The specifications of the 802.15.4 standards are available on the IEEE 802.15 working group website [5]
- **2G/ 3G/ 4G - Mobile Communication :** There are different generations of mobile communication standards including second generation (2G including GSM and CDMA), third generation (3G - including UMTS and CDMA2000) and fourth generation (4G - including LTE). IoT devices based on these standards can communicate over cellular networks. Data rates for these standards range from 9.6 Kb/s (for 2G) to upto 100 Mb/s (for 4G) and are available from the 3GPP websites.

Network/Internet Layer

The network layers are responsible for sending of IP datagrams from the source network to the destination network. This layer performs the host addressing and packet routing. The datagrams contain the source and destination addresses which are used to route them from the source to destination across multiple networks. Host identification is done using hierarchical IP addressing schemes such as IPv4 or IPv6.

- **IPv4 :** Internet Protocol version 4 (IPv4) is the most deployed Internet protocol that is used to identify the devices on a network using a hierarchical addressing scheme. IPv4 uses a 32-bit address scheme that allows total of 2^{32} or 4,294,967,296 addresses. As more and more devices got connected to the Internet, these addresses got exhausted in the year 2011. IPv4 has been succeeded by IPv6. The IP protocols establish connections on packet networks, but do not guarantee delivery of packets. Guaranteed delivery and data integrity are handled by the upper layer protocols (such as TCP). IPv4 is formally described in RFC 791 [6].
- **IPv6 :** Internet Protocol version 6 (IPv6) is the newest version of Internet protocol and successor to IPv4. IPv6 uses 128-bit address scheme that allows total of 2^{128} or 3.4×10^{38} addresses. IPv4 is formally described in RFC 2460 [7].
- **6LoWPAN :** 6LoWPAN (IPv6 over Low power Wireless Personal Area Networks) brings IP protocol to the low-power devices which have limited processing capability. 6LoWPAN operates in the 2.4 GHz frequency range and provides data transfer rates of 250 Kb/s. 6LoWPAN works with the 802.15.4 link layer protocol and defines compression mechanisms for IPv6 datagrams over IEEE 802.15.4-based networks [8].

Transport Layer

The transport layer protocols provide end-to-end message transfer capability independent of the underlying network. The message transfer capability can be set up on connections, either using handshakes (as in TCP) or without handshakes/acknowledgements (as in UDP). The transport layer provides functions such as error control, segmentation, flow control and congestion control.

- **TCP :** Transmission Control Protocol (TCP) is the most widely used transport layer protocol, that is used by web browsers (along with HTTP, HTTPS application layer protocols), email programs (SMTP application layer protocol) and file transfer (FTP). TCP is a connection oriented and stateful protocol. While IP protocol deals with sending packets, TCP ensures reliable transmission of packets in-order. TCP also provides error detection capability so that duplicate packets can be discarded and lost packets are retransmitted. The flow control capability of TCP ensures that

rate at which the sender sends the data is not too high for the receiver to process. The congestion control capability of TCP helps in avoiding network congestion and congestion collapse which can lead to degradation of network performance. TCP is described in RFC 793 [9].

- **UDP :** Unlike TCP, which requires carrying out an initial setup procedure, UDP is a connectionless protocol. UDP is useful for time-sensitive applications that have very small data units to exchange and do not want the overhead of connection setup. UDP is a transaction oriented and stateless protocol. UDP does not provide guaranteed delivery, ordering of messages and duplicate elimination. Higher levels of protocols can ensure reliable delivery or ensuring connections created are reliable. UDP is described in RFC 768 [10].

Application Layer

Application layer protocols define how the applications interface with the lower layer protocols to send the data over the network. The application data, typically in files, is encoded by the application layer protocol and encapsulated in the transport layer protocol which provides connection or transaction oriented communication over the network. Port numbers are used for application addressing (for example port 80 for HTTP, port 22 for SSH, etc.). Application layer protocols enable process-to-process connections using ports.

- **HTTP :** Hypertext Transfer Protocol (HTTP) is the application layer protocol that forms the foundation of the World Wide Web (WWW). HTTP includes commands such as GET, PUT, POST, DELETE, HEAD, TRACE, OPTIONS, etc. The protocol follows a request-response model where a client sends requests to a server using the HTTP commands. HTTP is a stateless protocol and each HTTP request is independent of the other requests. An HTTP client can be a browser or an application running on the client (e.g., an application running on an IoT device, a mobile application or other software). HTTP protocol uses Universal Resource Identifiers (URIs) to identify HTTP resources. HTTP is described in RFC 2616 [11].
- **CoAP :** Constrained Application Protocol (CoAP) is an application layer protocol for machine-to-machine (M2M) applications, meant for constrained environments with constrained devices and constrained networks. Like HTTP, CoAP is a web transfer protocol and uses a request-response model, however it runs on top of UDP instead of TCP. CoAP uses a client-server architecture where clients communicate with servers using connectionless datagrams. CoAP is designed to easily interface with HTTP. Like HTTP, CoAP supports methods such as GET, PUT, POST, and DELETE. CoAP draft specifications are available on IEFT Constrained environments (CoRE) Working Group website [12].

- **WebSocket :** WebSocket protocol allows full-duplex communication over a single socket connection for sending messages between client and server. WebSocket is based on TCP and allows streams of messages to be sent back and forth between the client and server while keeping the TCP connection open. The client can be a browser, a mobile application or an IoT device. WebSocket is described in RFC 6455 [13].
- **MQTT :** Message Queue Telemetry Transport (MQTT) is a light-weight messaging protocol based on the publish-subscribe model. MQTT uses a client-server architecture where the client (such as an IoT device) connects to the server (also called MQTT Broker) and publishes messages to topics on the server. The broker forwards the messages to the clients subscribed to topics. MQTT is well suited for constrained environments where the devices have limited processing and memory resources and the network bandwidth is low. MQTT specifications are available on IBM developerWorks [14].
- **XMPP :** Extensible Messaging and Presence Protocol (XMPP) is a protocol for real-time communication and streaming XML data between network entities. XMPP powers wide range of applications including messaging, presence, data syndication, gaming, multi-party chat and voice/video calls. XMPP allows sending small chunks of XML data from one network entity to another in near real-time. XMPP is a decentralized protocol and uses a client-server architecture. XMPP supports both client-to-server and server-to-server communication paths. In the context of IoT, XMPP allows real-time communication between IoT devices. XMPP is described in RFC 6120 [15].
- **DDS :** Data Distribution Service (DDS) is a data-centric middleware standard for device-to-device or machine-to-machine communication. DDS uses a publish-subscribe model where publishers (e.g. devices that generate data) create topics to which subscribers (e.g., devices that want to consume data) can subscribe. Publisher is an object responsible for data distribution and the subscriber is responsible for receiving published data. DDS provides quality-of-service (QoS) control and configurable reliability. DDS is described in Object Management Group (OMG) DDS specification [16].
- **AMQP :** Advanced Message Queuing Protocol (AMQP) is an open application layer protocol for business messaging. AMQP supports both point-to-point and publisher/subscriber models, routing and queuing. AMQP brokers receive messages from publishers (e.g., devices or applications that generate data) and route them over connections to consumers (applications that process data). Publishers publish the messages to exchanges which then distribute message copies to queues. Messages are either delivered by the broker to the consumers which have subscribed to the queues or the consumers can pull the messages from the queues. AMQP specification is

available on the AMQP working group website [17].

1.3 Logical Design of IoT

Logical design of an IoT system refers to an abstract representation of the entities and processes without going into the low-level specifics of the implementation. In this section we describe the functional blocks of an IoT system and the communication APIs that are used for the examples in this book. The steps in logical design are described in additional detail in Chapter-5.

1.3.1 IoT Functional Blocks

An IoT system comprises of a number of functional blocks that provide the system the capabilities for identification, sensing, actuation, communication, and management as shown in Figure 1.6. These functional blocks are described as follows:

- **Device :** An IoT system comprises of devices that provide sensing, actuation, monitoring and control functions. You learned about IoT devices in section 1.2.
- **Communication :** The communication block handles the communication for the IoT system. You learned about various protocols used for communication by IoT systems in section 1.2.
- **Services :** An IoT system uses various types of IoT services such as services for device monitoring, device control services, data publishing services and services for device discovery.
- **Management :** Management functional block provides various functions to govern the IoT system.
- **Security** Security functional block secures the IoT system and by providing functions such as authentication, authorization, message and content integrity, and data security.
- **Application :** IoT applications provide an interface that the users can use to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and view or analyze the processed data.

1.3.2 IoT Communication Models

- **Request-Response :** Request-Response is a communication model in which the client sends requests to the server and the server responds to the requests. When the server receives a request, it decides how to respond, fetches the data, retrieves resource representations, prepares the response, and then sends the response to the client. Request-Response model is a stateless communication model and each request-response pair is independent of others. Figure 1.7 shows the client-server

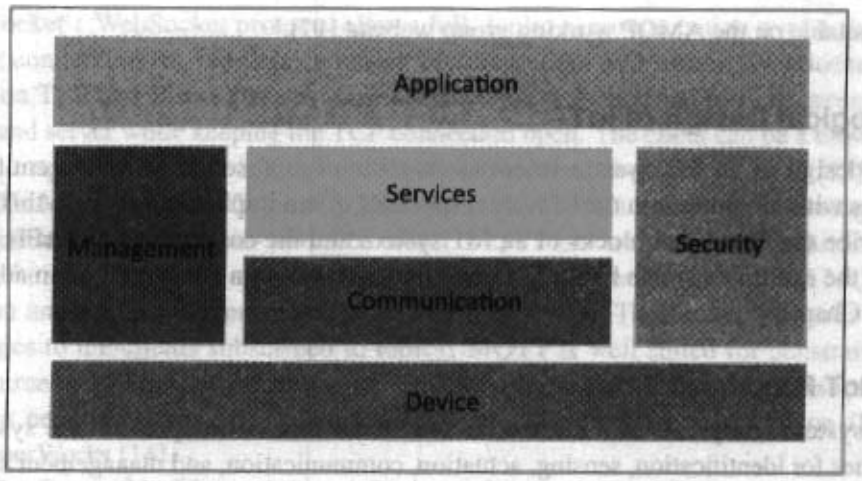


Figure 1.6: Functional Blocks of IoT

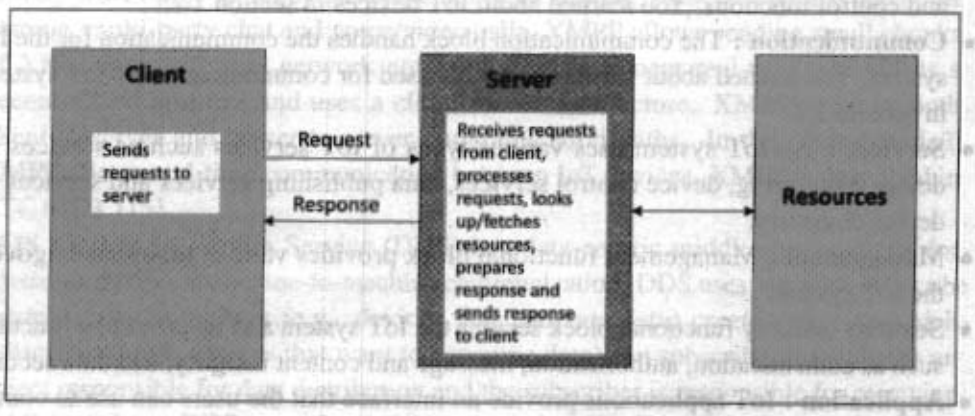


Figure 1.7: Request-Response communication model

interactions in the request-response model.

- **Publish-Subscribe :** Publish-Subscribe is a communication model that involves publishers, brokers and consumers. Publishers are the source of data. Publishers send the data to the topics which are managed by the broker. Publishers are not aware of the consumers. Consumers subscribe to the topics which are managed by the broker. When the broker receives data for a topic from the publisher, it sends the data to all the subscribed consumers. Figure 1.8 shows the publisher-broker-consumer interactions

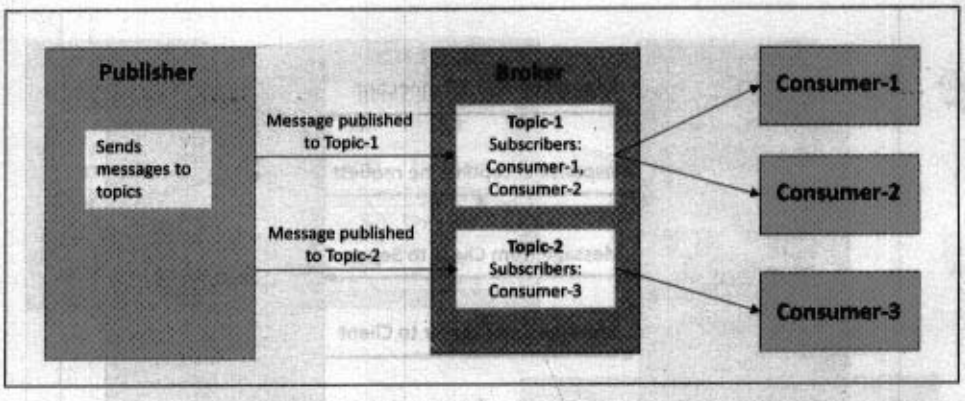


Figure 1.8: Publish-Subscribe communication model

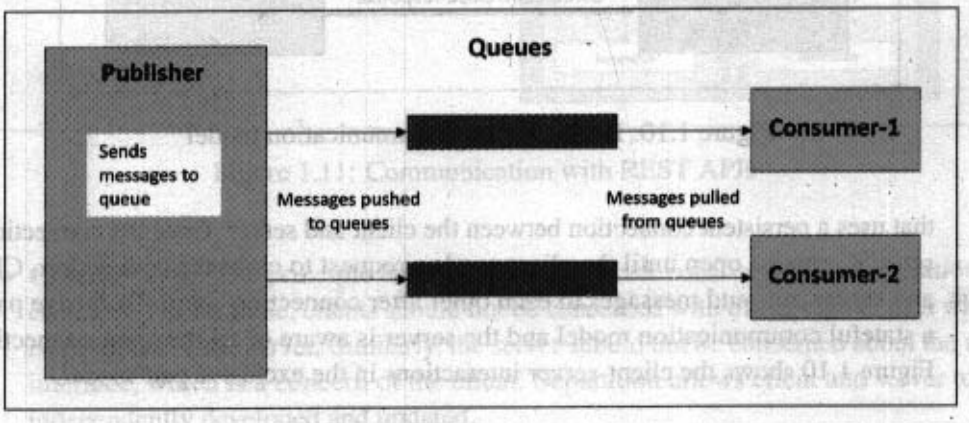


Figure 1.9: Push-Pull communication model

in the publish-subscribe model.

- **Push-Pull :** Push-Pull is a communication model in which the data producers push the data to queues and the consumers pull the data from the queues. Producers do not need to be aware of the consumers. Queues help in decoupling the messaging between the producers and consumers. Queues also act as a buffer which helps in situations when there is a mismatch between the rate at which the producers push data and the rate rate at which the consumers pull data. Figure 1.9 shows the publisher-queue-consumer interactions in the push-pull model.
- **Exclusive Pair :** Exclusive Pair is a bi-directional, fully duplex communication model

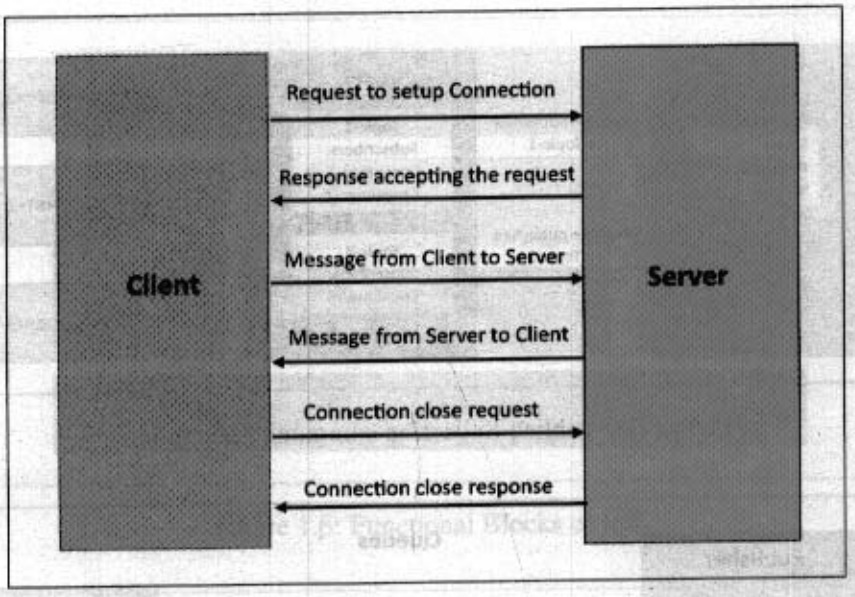


Figure 1.10: Exclusive Pair communication model

that uses a persistent connection between the client and server. Once the connection is setup it remains open until the client sends a request to close the connection. Client and server can send messages to each other after connection setup. Exclusive pair is a stateful communication model and the server is aware of all the open connections. Figure 1.10 shows the client-server interactions in the exclusive pair model.

1.3.3 IoT Communication APIs

In the previous section you learned about various communication models. In this section you will learn about two specific communication APIs which are used in the examples in this book.

REST-based Communication APIs

Representational State Transfer (REST) [88] is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred. REST APIs follow the request-response communication model described in previous section. The REST architectural constraints apply to the components, connectors, and data elements, within a distributed hypermedia system. The REST architectural constraints are as follows:

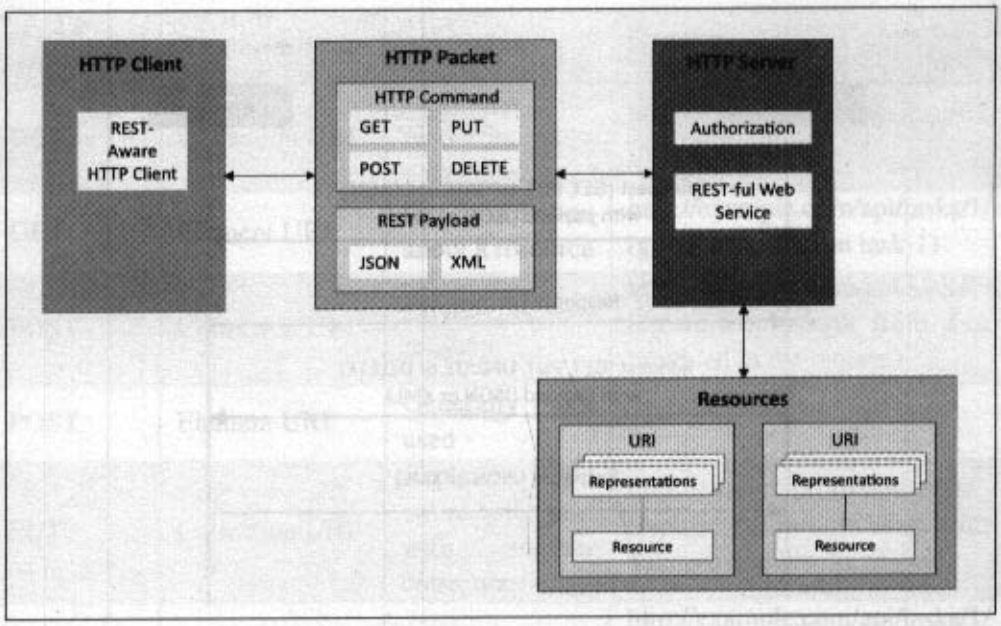


Figure 1.11: Communication with REST APIs

- **Client-Server**: The principle behind the client-server constraint is the separation of concerns. For example, clients should not be concerned with the storage of data which is a concern of the server. Similarly, the server should not be concerned about the user interface, which is a concern of the client. Separation allows client and server to be independently developed and updated.
- **Stateless**: Each request from client to server must contain all the information necessary to understand the request, and cannot take advantage of any stored context on the server. The session state is kept entirely on the client.
- **Cache-able**: Cache constraint requires that the data within a response to a request be implicitly or explicitly labeled as cache-able or non-cache-able. If a response is cache-able, then a client cache is given the right to reuse that response data for later, equivalent requests. Caching can partially or completely eliminate some interactions and improve efficiency and scalability.
- **Layered System**: Layered system constraint, constrains the behavior of components such that each component cannot see beyond the immediate layer with which they are interacting. For example, a client cannot tell whether it is connected directly to the end server, or to an intermediary along the way. System scalability can be improved

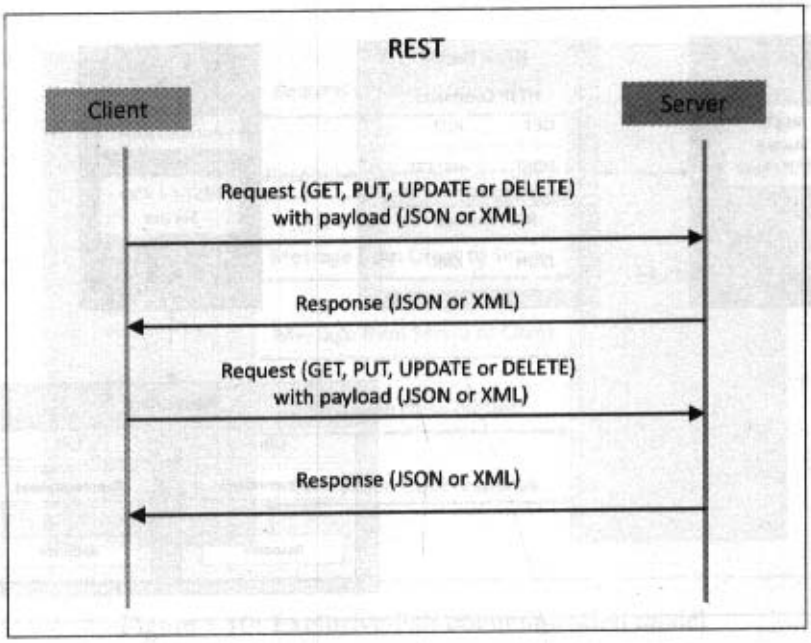


Figure 1.12: Request-response model used by REST

by allowing intermediaries to respond to requests instead of the end server, without the client having to do anything different.

- **Uniform Interface**: Uniform Interface constraint requires that the method of communication between a client and a server must be uniform. Resources are identified in the requests (by URIs in web based systems) and are themselves separate from the representations of the resources that are returned to the client. When a client holds a representation of a resource it has all the information required to update or delete the resource (provided the client has required permissions). Each message includes enough information to describe how to process the message.
- **Code on demand**: Servers can provide executable code or scripts for clients to execute in their context. This constraint is the only one that is optional.

A RESTful web service is a "web API" implemented using HTTP and REST principles. Figure 1.11 shows the communication between client and server using REST APIs. Figure 1.12 shows the interactions in the request-response model used by REST. RESTful web service is a collection of resources which are represented by URIs. RESTful web API has a base URI (e.g. http://example.com/api/tasks/). The clients send requests to these URIs using the

HTTP Method	Resource Type	Action	Example
GET	Collection URI	List all the resources in a collection	http://example.com/api/tasks/ (list all tasks)
GET	Element URI	Get information about a resource	http://example.com/api/tasks/1/ (get information on task-1)
POST	Collection URI	Create a new resource	http://example.com/api/tasks/ (create a new task from data provided in the request)
POST	Element URI	Generally not used	
PUT	Collection URI	Replace the entire collection with another collection	http://example.com/api/tasks/ (replace entire collection with data provided in the request)
PUT	Element URI	Update a resource	http://example.com/api/tasks/1/ (update task-1 with data provided in the request)
DELETE	Collection URI	Delete the entire collection	http://example.com/api/tasks/ (delete all tasks)
DELETE	Element URI	Delete a resource	http://example.com/api/tasks/1/ (delete task-1)

Table 1.1: HTTP request methods and actions

methods defined by the HTTP protocol (e.g., GET, PUT, POST, or DELETE), as shown in Table 1.1. A RESTful web service can support various Internet media types (JSON being the most popular media type for RESTful web services). IP for Smart Objects Alliance (IPSO Alliance) has published an Application Framework that defines a RESTful design for use in IP smart object systems [18].

WebSocket-based Communication APIs

WebSocket APIs allow bi-directional, full duplex communication between clients and servers. WebSocket APIs follow the exclusive pair communication model described in previous section and as shown in Figure 1.13. Unlike request-response APIs such as REST, the

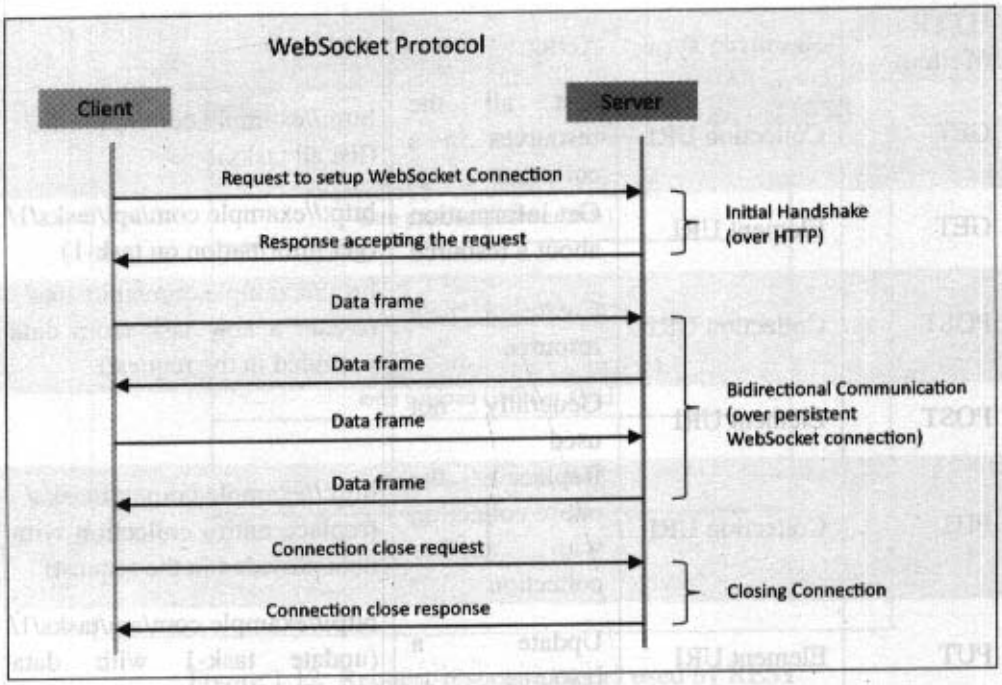


Figure 1.13: Exclusive pair model used by WebSocket APIs

WebSocket APIs allow full duplex communication and do not require a new connection to be setup for each message to be sent. WebSocket communication begins with a connection setup request sent by the client to the server. This request (called a WebSocket handshake) is sent over HTTP and the server interprets it as an upgrade request. If the server supports WebSocket protocol, the server responds to the WebSocket handshake response. After the connection is setup, the client and server can send data/messages to each other in full-duplex mode. WebSocket APIs reduce the network traffic and latency as there is no overhead for connection setup and termination requests for each message. WebSocket is suitable for IoT applications that have low latency or high throughput requirements.

1.4 IoT Enabling Technologies

IoT is enabled by several technologies including wireless sensor networks, cloud computing, big data analytics, embedded systems, security protocols and architectures, communication protocols, web services, mobile Internet, and semantic search engines. This section provides an overview of some of these technologies which play a key-role in IoT.

1.4.1 Wireless Sensor Networks

A Wireless Sensor Network (WSN) comprises of distributed devices with sensors which are used to monitor the environmental and physical conditions. A WSN consist of a number of end-nodes and routers and a coordinator. End nodes have several sensors attached to them. End nodes can also act as routers. Routers are responsible for routing the data packets from end-nodes to the coordinator. The coordinator collects the data from all the nodes. Coordinator also acts as a gateway that connects the WSN to the Internet. Some examples of WSNs used in IoT systems are described as follows:

- Weather monitoring systems use WSNs in which the nodes collect temperature, humidity and other data, which is aggregated and analyzed.
- Indoor air quality monitoring systems use WSNs to collect data on the indoor air quality and concentration of various gases.
- Soil moisture monitoring systems use WSNs to monitor soil moisture at various locations.
- Surveillance systems use WSNs for collecting surveillance data (such as motion detection data)
- Smart grids use WSNs for monitoring the grid at various points.
- Structural health monitoring systems use WSNs to monitor the health of structures (buildings, bridges) by collecting vibration data from sensor nodes deployed at various points in the structure.

WSNs are enabled by wireless communication protocols such as IEEE 802.15.4. ZigBee is one of the most popular wireless technologies used by WSNs. ZigBee specifications are based on IEEE 802.15.4. ZigBee operates at 2.4 GHz frequency and offers data rates upto 250 KB/s and range from 10 to 100 meters depending on the power output and environmental conditions. The power of WSNs lies in their ability to deploy large number of low-cost and low-power sensing nodes for continuous monitoring of environmental and physical conditions. WSNs are self-organizing networks. Since WSNs have large number of nodes, manual configuration for each node is not possible. The self-organizing capability of WSN makes the network robust. In the event of failure of some nodes or addition of new nodes to the network, the network can reconfigure itself.

1.4.2 Cloud Computing

Cloud computing is a transformative computing paradigm that involves delivering applications and services over the Internet. Cloud computing involves provisioning of computing, networking and storage resources on demand and providing these resources as metered services to the users, in a "pay as you go" model. Cloud computing resources can be provisioned on-demand by the users, without requiring interactions with the cloud service

provider. The process of provisioning resources is automated. Cloud computing resources can be accessed over the network using standard access mechanisms that provide platform-independent access through the use of heterogeneous client platforms such as workstations, laptops, tablets and smart-phones. The computing and storage resources provided by cloud service providers are pooled to serve multiple users using multi-tenancy. Multi-tenant aspects of the cloud allow multiple users to be served by the same physical hardware. Users are assigned virtual resources that run on top of the physical resources.

Cloud computing services are offered to users in different forms (see the authors' companion book on Cloud Computing, for instance):

- **Infrastructure-as-a-Service (IaaS) :** IaaS provides the users the ability to provision computing and storage resources. These resources are provided to the users as virtual machine instances and virtual storage. Users can start, stop, configure and manage the virtual machine instances and virtual storage. Users can deploy operating systems and applications of their choice on the virtual resources provisioned in the cloud. The cloud service provider manages the underlying infrastructure. Virtual resources provisioned by the users are billed based on a pay-per-use paradigm.
- **Platform-as-a-Service (PaaS) :** PaaS provides the users the ability to develop and deploy application in the cloud using the development tools, application programming interfaces (APIs), software libraries and services provided by the cloud service provider. The cloud service provider manages the underlying cloud infrastructure including servers, network, operating systems and storage. The users, themselves, are responsible for developing, deploying, configuring and managing applications on the cloud infrastructure.
- **Software-as-a-Service (SaaS) :** SaaS provides the users a complete software application or the user interface to the application itself. The cloud service provider manages the underlying cloud infrastructure including servers, network, operating systems, storage and application software, and the user is unaware of the underlying architecture of the cloud. Applications are provided to the user through a thin client interface (e.g., a browser). SaaS applications are platform independent and can be accessed from various client devices such as workstations, laptop, tablets and smart-phones, running different operating systems. Since the cloud service provider manages both the application and data, the users are able to access the applications from anywhere.

1.4.3 Big Data Analytics

Big data is defined as collections of data sets whose volume, velocity (in terms of its temporal variation), or variety, is so large that it is difficult to store, manage, process and analyze the data using traditional databases and data processing tools. Big data analytics involves

several steps starting from data cleansing, data munging (or wrangling), data processing and visualization. Some examples of big data generated by IoT systems are described as follows:

- Sensor data generated by IoT systems such as weather monitoring stations.
- Machine sensor data collected from sensors embedded in industrial and energy systems for monitoring their health and detecting failures.
- Health and fitness data generated by IoT devices such as wearable fitness bands.
- Data generated by IoT systems for location and tracking of vehicles.
- Data generated by retail inventory monitoring systems.

The underlying characteristics of big data include:

- **Volume:** Though there is no fixed threshold for the volume of data to be considered as big data, however, typically, the term big data is used for massive scale data that is difficult to store, manage and process using traditional databases and data processing architectures. The volumes of data generated by modern IT, industrial, and health-care systems, for example, is growing exponentially driven by the lowering costs of data storage and processing architectures and the need to extract valuable insights from the data to improve business processes, efficiency and service to consumers.
- **Velocity:** Velocity is another important characteristic of big data and the primary reason for exponential growth of data. Velocity of data refers to how fast the data is generated and how frequently it varies. Modern IT, industrial and other systems are generating data at increasingly higher speeds.
- **Variety:** Variety refers to the forms of the data. Big data comes in different forms such as structured or unstructured data, including text data, image, audio, video and sensor data.

1.4.4 Communication Protocols

Communication protocols form the backbone of IoT systems and enable network connectivity and coupling to applications. Communication protocols allow devices to exchange data over the network. In section 1.2.2 you learned about various link, network, transport and application layer protocols. These protocols define the data exchange formats, data encoding, addressing schemes for devices and routing of packets from source to destination. Other functions of the protocols include sequence control (that helps in ordering packets determining lost packets), flow control (that helps in controlling the rate at which the sender is sending the data so that the receiver or the network is not overwhelmed) and retransmission of lost packets.

1.4.5 Embedded Systems

An Embedded System is a computer system that has computer hardware and software embedded to perform specific tasks. In contrast to general purpose computers or personal computers (PCs) which can perform various types of tasks, embedded systems are designed to perform a specific set of tasks. Key components of an embedded system include, microprocessor or microcontroller, memory (RAM, ROM, cache), networking units (Ethernet, WiFi adapters), input/output units (display, keyboard, etc.) and storage (such as flash memory). Some embedded systems have specialized processors such as digital signal processors (DSPs), graphics processors and application specific processors. Embedded systems run embedded operating systems such as real-time operating systems (RTOS). Embedded systems range from low-cost miniaturized devices such as digital watches to devices such as digital cameras, point of sale terminals, vending machines, appliances (such as washing machines), etc. In the next chapter we describe how such devices form an integral part of IoT systems.

1.5 IoT Levels & Deployment Templates

In this section we define various levels of IoT systems with increasing completely. An IoT system comprises of the following components:

- **Device:** An IoT device allows identification, remote sensing, actuating and remote monitoring capabilities. You learned about various examples of IoT devices in section 1.2.1.
- **Resource:** Resources are software components on the IoT device for accessing, processing, and storing sensor information, or controlling actuators connected to the device. Resources also include the software components that enable network access for the device.
- **Controller Service:** Controller service is a native service that runs on the device and interacts with the web services. Controller service sends data from the device to the web service and receives commands from the application (via web services) for controlling the device.
- **Database:** Database can be either local or in the cloud and stores the data generated by the IoT device.
- **Web Service:** Web services serve as a link between the IoT device, application, database and analysis components. Web service can be either implemented using HTTP and REST principles (REST service) or using WebSocket protocol (WebSocket service). A comparison of REST and WebSocket is provided below:

- **Stateless/Stateful:** REST services are stateless in nature. Each request contains all the information needed to process it. Requests are independent of each other. WebSocket on the other hand is stateful in nature where the server maintains the state and is aware of all the open connections.
- **Uni-directional/Bi-directional:** REST services operate over HTTP and are uni-directional. Request is always sent by a client and the server responds to the requests. On the other hand, WebSocket is a bi-directional protocol and allows both client and server to send messages to each other.
- **Request-Response/Full Duplex:** REST services follow a request-response communication model where the client sends requests and the server responds to the requests. WebSocket on the other hand allow full-duplex communication between the client and server, i.e., both client and server can send messages to each other independently.
- **TCP Connections:** For REST services, each HTTP request involves setting up a new TCP connection. WebSocket on the other hand involves a single TCP connection over which the client and server communicate in a full-duplex mode.
- **Header Overhead:** REST services operate over HTTP, and each request is independent of others. Thus each request carries HTTP headers which is an overhead. Due the overhead of HTTP headers, REST is not suitable for real-time applications. WebSocket on the other hand does not involve overhead of headers. After the initial handshake (that happens over HTTP), the client and server exchange messages with minimal frame information. Thus WebSocket is suitable for real-time applications.
- **Scalability:** Scalability is easier in the case of REST services as requests are independent and no state information needs to be maintained by the server. Thus both horizontal (scaling-out) and vertical scaling (scaling-up) solutions are possible for REST services. For WebSockets, horizontal scaling can be cumbersome due to the stateful nature of the communication. Since the server maintains the state of a connection, vertical scaling is easier for WebSockets than horizontal scaling.

- **Analysis Component:** The Analysis Component is responsible for analyzing the IoT data and generate results in a form which are easy for the user to understand. Analysis of IoT data can be performed either locally or in the cloud. Analyzed results are stored in the local or cloud databases.
- **Application:** IoT applications provide an interface that the users can use to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and view the processed data.

1.5.1 IoT Level-1

A level-1 IoT system has a single node/device that performs sensing and/or actuation, stores data, performs analysis and hosts the application as shown in Figure 1.14. Level-1 IoT systems are suitable for modeling low-cost and low-complexity solutions where the data involved is not big and the analysis requirements are not computationally intensive.

Let us now consider an example of a level-1 IoT system for home automation. The system consists of a single node that allows controlling the lights and appliances in a home remotely. The device used in this system interfaces with the lights and appliances using electronic relay switches. The status information of each light or appliance is maintained in a local database. REST services deployed locally allow retrieving and updating the state of each light or appliance in the status database. The controller service continuously monitors the state of each light or appliance (by retrieving state from the database) and triggers the relay switches accordingly. The application which is deployed locally has a user interface for controlling the lights or appliances. Since the device is connected to the Internet, the application can be accessed remotely as well.

1.5.2 IoT Level-2

A level-2 IoT system has a single node that performs sensing and/or actuation and local analysis as shown in Figure 1.15. Data is stored in the cloud and application is usually cloud-based. Level-2 IoT systems are suitable for solutions where the data involved is big, however, the primary analysis requirement is not computationally intensive and can be done locally itself.

Let us consider an example of a level-2 IoT system for smart irrigation. The system consists of a single node that monitors the soil moisture level and controls the irrigation system. The device used in this system collects soil moisture data from sensors. The controller service continuously monitors the moisture levels. If the moisture level drops below a threshold, the irrigation system is turned on. For controlling the irrigation system actuators such as solenoid valves can be used. The controller also sends the moisture data to the computing cloud. A cloud-based REST web service is used for storing and retrieving moisture data which is stored in the cloud database. A cloud-based application is used for visualizing the moisture levels over a period of time, which can help in making decisions about irrigation schedules.

1.5.3 IoT Level-3

A level-3 IoT system has a single node. Data is stored and analyzed in the cloud and application is cloud-based as shown in Figure 1.16. Level-3 IoT systems are suitable for

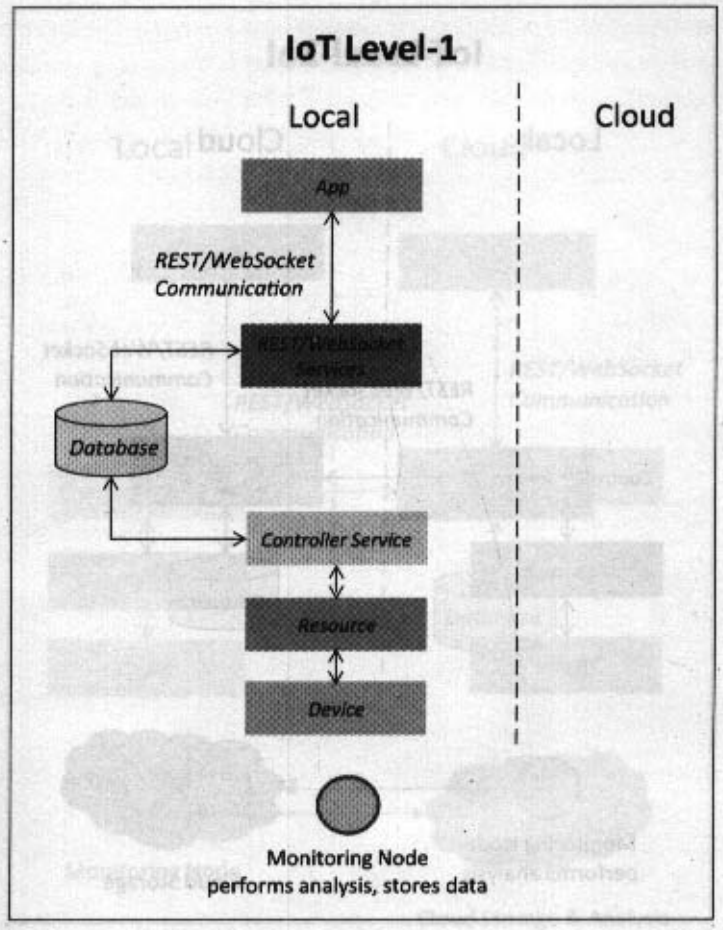


Figure 1.14: IoT Level-1

solutions where the data involved is big and the analysis requirements are computationally intensive.

Let us consider an example of a level-2 IoT system for tracking package handling. The system consists of a single node (for a package) that monitors the vibration levels for a package being shipped. The device in this system uses accelerometer and gyroscope sensors for monitoring vibration levels. The controller service sends the sensor data to the cloud in real-time using a WebSocket service. The data is stored in the cloud and also visualized using a cloud-based application. The analysis components in the cloud can trigger alerts if

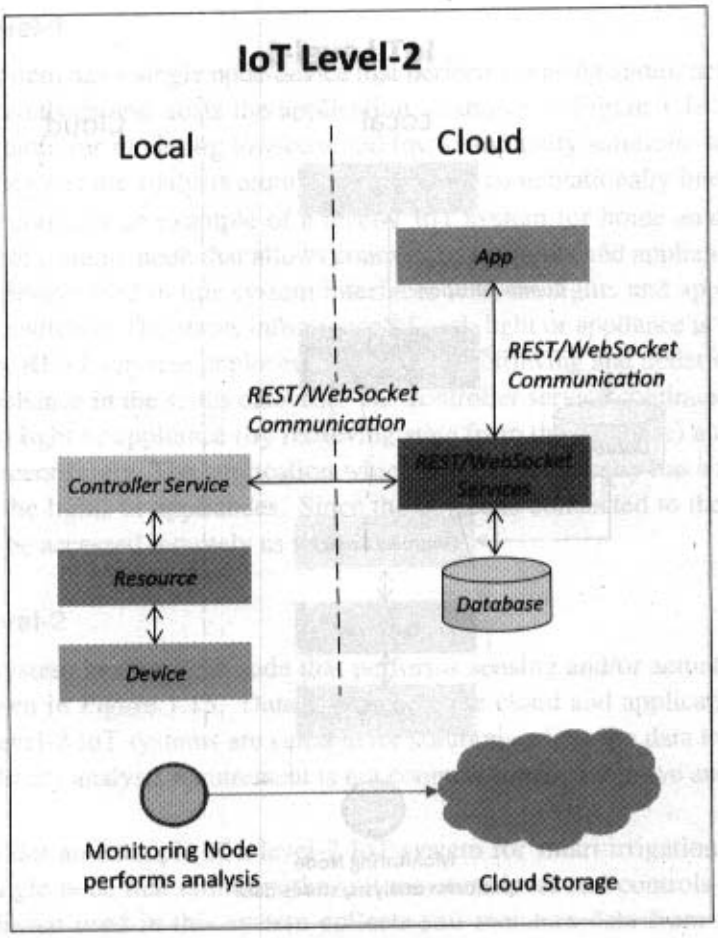


Figure 1.15: IoT Level-2

the vibration levels become greater than a threshold. The benefit of using WebSocket service instead of REST service in this example is that, the sensor data can be sent in real time to the cloud. Moreover, cloud based applications can subscribe to the sensor data feeds for viewing the real-time data.

1.5.4 IoT Level-4

A level-4 IoT system has multiple nodes that perform local analysis. Data is stored in the cloud and application is cloud-based as shown in Figure 1.17. Level-4 contains local and

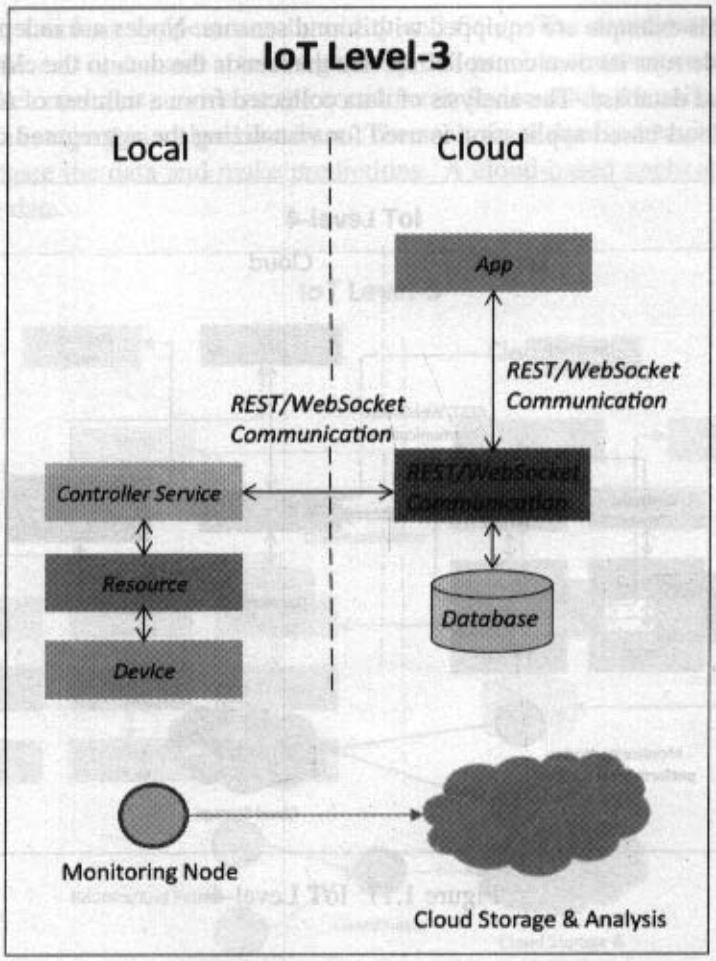


Figure 1.16: IoT Level-3

cloud-based observer nodes which can subscribe to and receive information collected in the cloud from IoT devices. Observer nodes can process information and use it for various applications, however, observer nodes do not perform any control functions. Level-4 IoT systems are suitable for solutions where multiple nodes are required, the data involved is big and the analysis requirements are computationally intensive.

Let us consider an example of a level-4 IoT system for noise monitoring. The system consists of multiple nodes placed in different locations for monitoring noise levels in an area.

The nodes in this example are equipped with sound sensors. Nodes are independent of each other. Each node runs its own controller service that sends the data to the cloud. The data is stored in a cloud database. The analysis of data collected from a number of nodes is done in the cloud. A cloud-based application is used for visualizing the aggregated data.

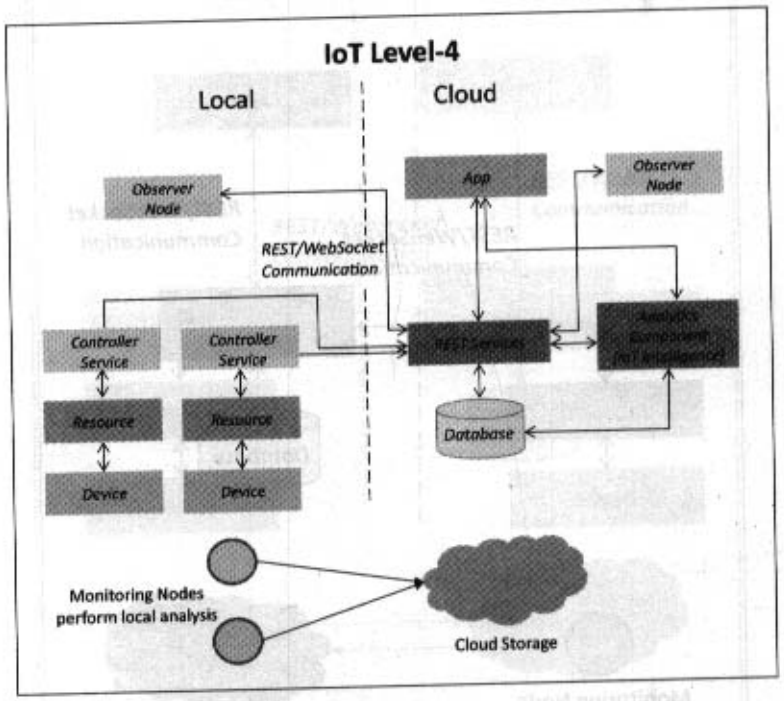


Figure 1.17: IoT Level-4

1.5.5 IoT Level-5

A level-5 IoT system has multiple end nodes and one coordinator node as shown in Figure 1.18. The end nodes that perform sensing and/or actuation. Coordinator node collects data from the end nodes and sends to the cloud. Data is stored and analyzed in the cloud and application is cloud-based. Level-5 IoT systems are suitable for solutions based on wireless sensor networks, in which the data involved is big and the analysis requirements are computationally intensive.

Let us consider an example of a level-5 IoT system for forest fire detection. The system consists of multiple nodes placed in different locations for monitoring temperature, humidity and carbon dioxide (CO_2) levels in a forest. The end nodes in this example are equipped with

various sensors (such as temperature, humidity and CO_2). The coordinator node collects the data from the end nodes and acts as a gateway that provides Internet connectivity to the IoT system. The controller service on the coordinator device sends the collected data to the cloud. The data is stored in a cloud database. The analysis of data is done in the computing cloud to aggregate the data and make predictions. A cloud-based application is used for visualizing the data.

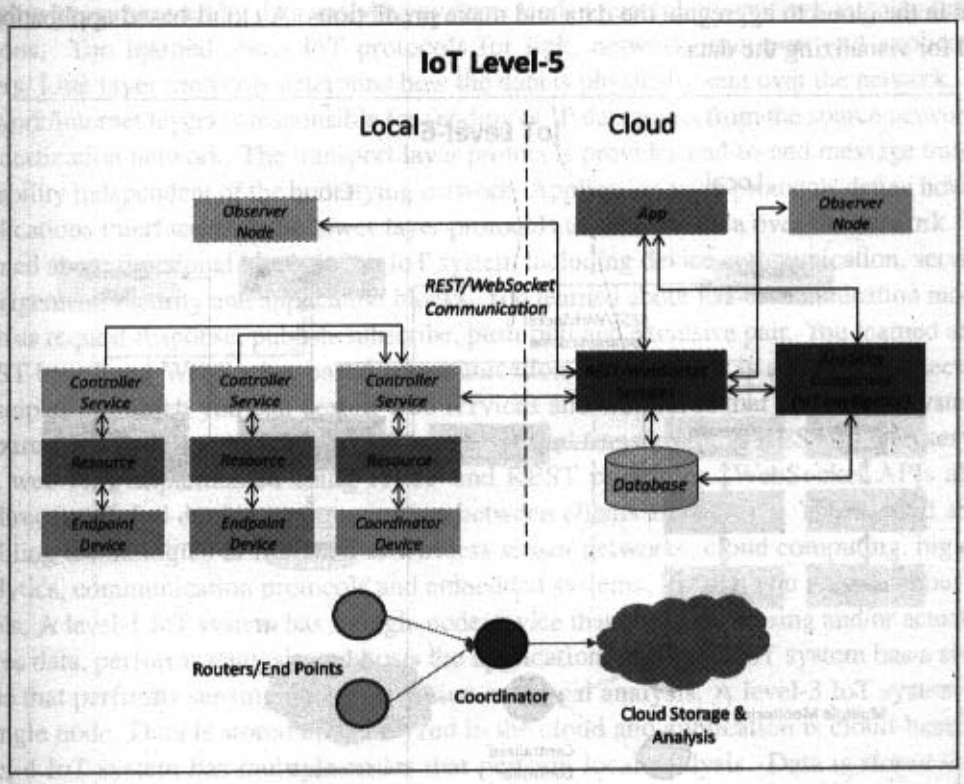


Figure 1.18: IoT Level-5

1.5.6 IoT Level-6

A level-6 IoT system has multiple independent end nodes that perform sensing and/or actuation and send data to the cloud. Data is stored in the cloud and application is cloud-based as shown in Figure 1.19. The analytics component analyzes the data and stores the results in the cloud database. The results are visualized with the cloud-based application. The centralized controller is aware of the status of all the end nodes and sends control commands

to the nodes.

Let us consider an example of a level-6 IoT system for weather monitoring. The system consists of multiple nodes placed in different locations for monitoring temperature, humidity and pressure in an area. The end nodes are equipped with various sensors (such as temperature, pressure and humidity). The end nodes send the data to the cloud in real-time using a WebSocket service. The data is stored in a cloud database. The analysis of data is done in the cloud to aggregate the data and make predictions. A cloud-based application is used for visualizing the data.

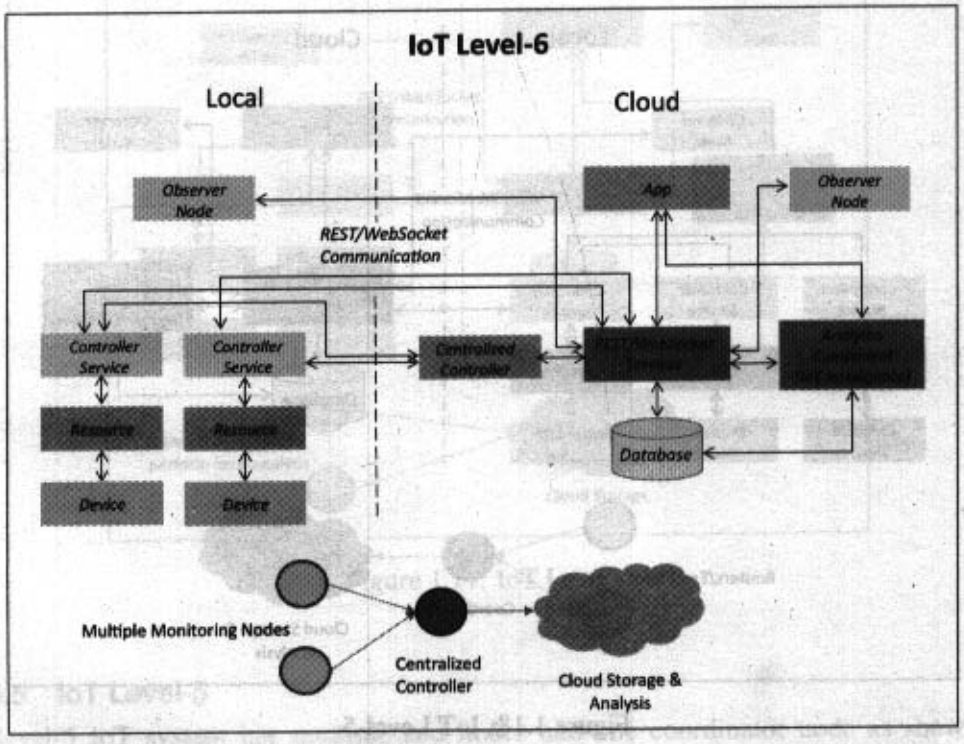


Figure 1.19: IoT Level-6

Summary

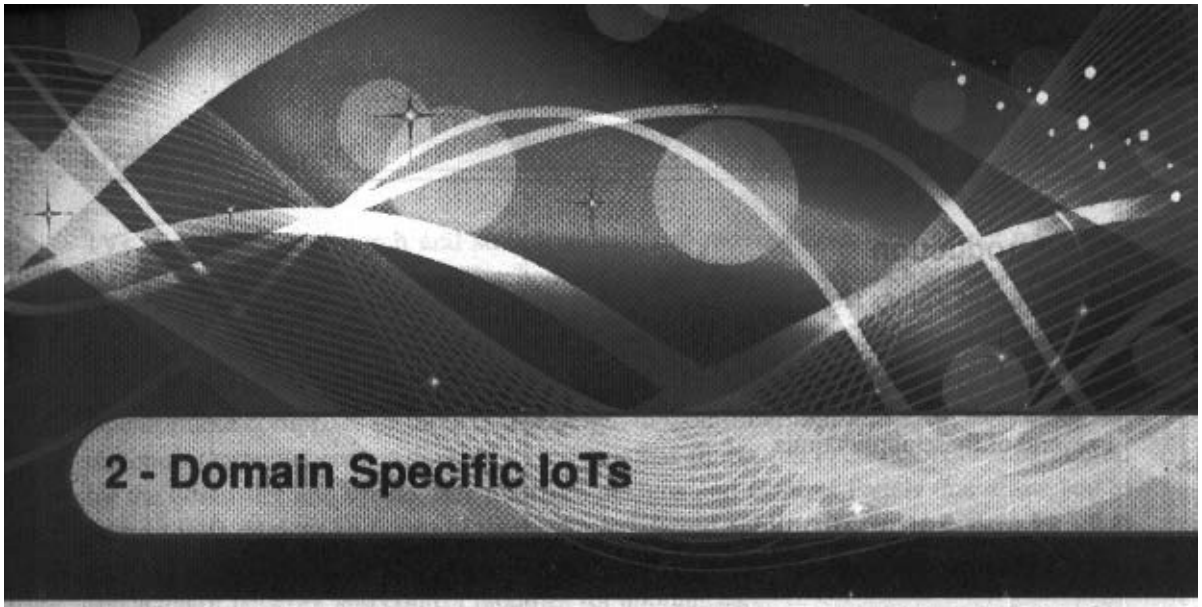
Internet of Things (IoT) refers to physical and virtual objects that have unique identities and are connected to the Internet. This allows the development of intelligent applications that make energy, logistics, industrial control, retail, agriculture and many other domains of

human endeavour "smarter". IoT allows different types of devices, appliances, users and machines to communicate and exchange data. The applications of Internet of Things (IoT) span a wide range of domains including (but not limited to) homes, cities, environment, energy systems, retail, logistics, industry, agriculture and health. Things in IoT refers to IoT devices which have unique identities and allow remote sensing, actuating and remote monitoring capabilities. Almost all IoT devices generate data in some form or the other which when processed by data analytics systems leads to useful information to guide further actions. You learned about IoT protocols for link, network, transport and application layers. Link layer protocols determine how the data is physically sent over the network. The network/internet layers is responsible for sending of IP datagrams from the source network to the destination network. The transport layer protocols provides end-to-end message transfer capability independent of the underlying network. Application layer protocols define how the applications interface with the lower layer protocols to send the data over the network. You learned about functional blocks of an IoT system including device communication, services, management, security and application blocks. You learned about IoT communication models such as request-response, publish-subscribe, push-pull and exclusive pair. You learned about REST-based and WebSocket-based communication APIs. REST is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred. A RESTful web service is a web API implemented using HTTP and REST principles. WebSocket APIs allow bi-directional, full duplex communication between clients and servers. You learned about enabling technologies of IoT such as wireless sensor networks, cloud computing, big data analytics, communication protocols and embedded systems. Finally, you learned about IoT levels. A level-1 IoT system has a single node/device that performs sensing and/or actuation, stores data, performs analysis and hosts the application. A level-2 IoT system has a single node that performs sensing and/or actuation and local analysis. A level-3 IoT system has a single node. Data is stored and analyzed in the cloud and application is cloud-based. A level-4 IoT system has multiple nodes that perform local analysis. Data is stored in the cloud and application is cloud-based. A level-5 IoT system has multiple end nodes and one coordinator node. A level-6 IoT system has multiple independent end nodes that perform sensing and/or actuation and send data to the cloud.

Review Questions

1. Describe an example of an IoT system in which information and knowledge are inferred from data.
2. Why do IoT systems have to be self-adapting and self-configuring?

3. What is the role of things and Internet in IoT?
4. What is the function of communication functional block in an IoT system?
5. Describe an example of IoT service that uses publish-subscribe communication model.
6. Describe an example of IoT service that uses WebSocket-based communication.
7. What are the architectural constraints of REST?
8. What is the role of a coordinator in wireless sensor network?
9. What is the role of a controller service in an IoT system?



2 - Domain Specific IoTs

This Chapter Covers

IoT Applicatins for:

- Home
- Cities
- Environment
- Energy Systems
- Retail
- Logistics
- Industry
- Agriculture
- Health & Lifestyle

2.1 Introduction

The Internet of Things (IoT) applications span a wide range of domains including (but not limited to) homes, cities, environment, energy systems, retail, logistics, industry, agriculture and health. This chapter provides an overview of various types of IoT applications for each of these domains. In the later chapters the reader is guided through detailed implementations of several of these applications.

2.2 Home Automation

2.2.1 Smart Lighting

Smart lighting for homes helps in saving energy by adapting the lighting to the ambient conditions and switching on/off or dimming the lights when needed. Key enabling technologies for smart lighting include solid state lighting (such as LED lights) and IP-enabled lights. For solid state lighting solutions both spectral and temporal characteristics can be configured to adapt illumination to various needs. Smart lighting solutions for home achieve energy savings by sensing the human movements and their environments and controlling the lights accordingly. Wireless-enabled and Internet connected lights can be controlled remotely from IoT applications such as a mobile or web application. Smart lights with sensors for occupancy, temperature, lux level, etc., can be configured to adapt the lighting (by changing the light intensity, color, etc.) based on the ambient conditions sensed, in order to provide a good ambiance. In [19] controllable LED lighting system is presented that is embedded with ambient intelligence gathered from a distributed smart wireless sensor network to optimize and control the lighting system to be more efficient and user-oriented. A solid state lighting model is described in [20] and implemented on a wireless sensor network that provides services for sensing illumination changes and dynamically adjusting luminary brightness according to user preferences. In chapter-9 we provide a case study on a smart lighting system.

2.2.2 Smart Appliances

Modern homes have a number of appliances such as TVs, refrigerators, music systems, washer/dryers, etc. Managing and controlling these appliances can be cumbersome, with each appliance having its own controls or remote controls. Smart appliances make the management easier and also provide status information to the users remotely. For example, smart washer/dryers that can be controlled remotely and notify when the washing/drying cycle is complete. Smart thermostats allow controlling the temperature remotely and can learn the user preferences [22]. Smart refrigerators can keep track of the items stored (using RFID tags) and send updates to the users when an item is low on stock. Smart

TVs allows users to search and stream videos and movies from the Internet on a local storage drive, search TV channel schedules and fetch news, weather updates and other content from the Internet. OpenRemote [21] is an open source automation platform for homes and buildings. OpenRemote is platform agnostic and works with standard hardware. With OpenRemote, users can control various appliances using mobile or web applications. OpenRemote comprises of three components - a Controller that manages scheduling and runtime integration between devices, a Designer that allows you to create both configurations for the controller and create user interface designs and Control Panels that allow you to interact with devices and control them. An IoT-based appliance control system for smart homes is described in [23], that uses a smart central controller to set up a wireless sensor and actuator network and control modules for appliances.

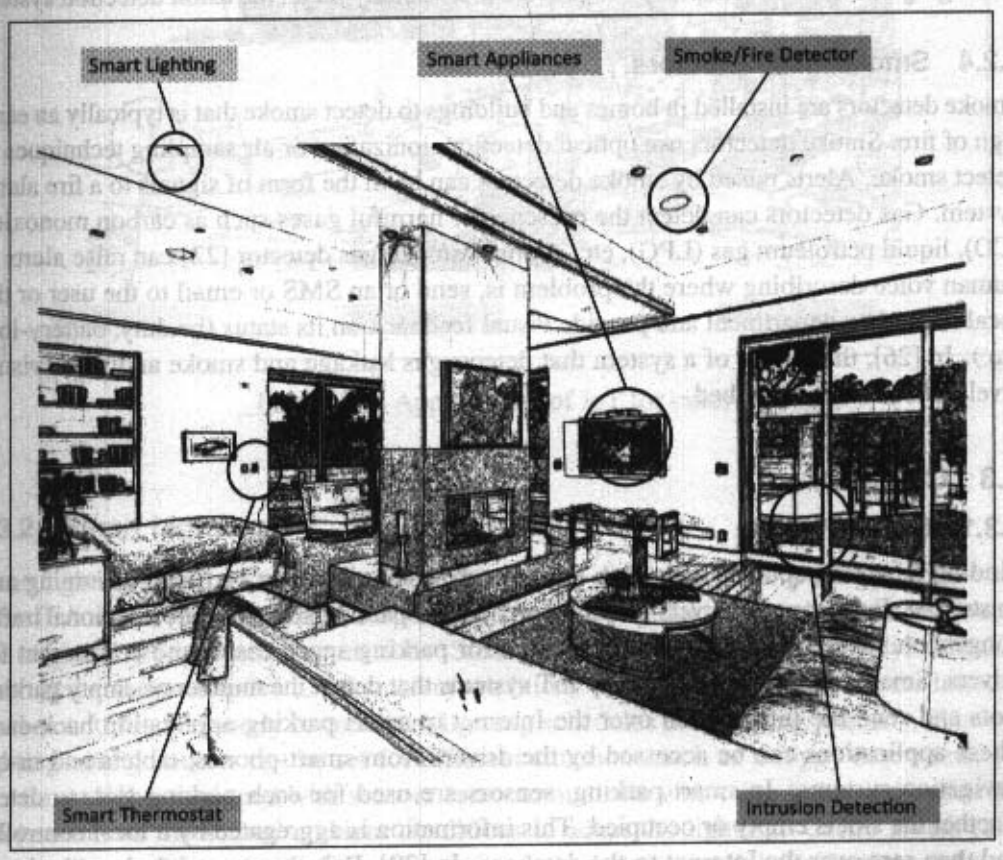


Figure 2.1: Applications of IoT for homes

2.2.3 Intrusion Detection

Home intrusion detection systems use security cameras and sensors (such as PIR sensors and door sensors) to detect intrusions and raise alerts. Alerts can be in the form of an SMS or an email sent to the user. Advanced systems can even send detailed alerts such as an image grab or a short video clip sent as an email attachment. A cloud controlled intrusion detection system is described in [24] that uses location-aware services, where the geo-location of each node of a home automation system is independently detected and stored in the cloud. In the event of intrusions, the cloud services alert the accurate neighbors (who are using the home automation system) or local police. In [25], an intrusion detection system based on UPnP technology is described. The system uses image processing to recognize the intrusion and extract the intrusion subject and generate Universal-Plug-and-Play (UPnP-based) instant messaging for alerts. In chapter-9 we provide a case study on an intrusion detection system.

2.2.4 Smoke/Gas Detectors

Smoke detectors are installed in homes and buildings to detect smoke that is typically an early sign of fire. Smoke detectors use optical detection, ionization or air sampling techniques to detect smoke. Alerts raised by smoke detectors can be in the form of signals to a fire alarm system. Gas detectors can detect the presence of harmful gases such as carbon monoxide (CO), liquid petroleum gas (LPG), etc. A smart smoke/gas detector [22] can raise alerts in human voice describing where the problem is, send or an SMS or email to the user or the local fire safety department and provide visual feedback on its status (healthy, battery-low, etc.). In [26], the design of a system that detects gas leakage and smoke and gives visual level indication, is described.

2.3 Cities

2.3.1 Smart Parking

Finding a parking space during rush hours in crowded cities can be time consuming and frustrating. Furthermore, drivers blindly searching for parking spaces create additional traffic congestion. Smart parking make the search for parking space easier and convenient for drivers. Smart parking are powered by IoT systems that detect the number of empty parking slots and send the information over the Internet to smart parking application back-ends. These applications can be accessed by the drivers from smart-phones, tablets and in-car navigation systems. In smart parking, sensors are used for each parking slot, to detect whether the slot is empty or occupied. This information is aggregated by a local controller and then sent over the Internet to the database. In [29], Polycarpou *et. al.* describe latest trends in parking availability monitoring, parking reservation and dynamic pricing schemes.

Design and implementation of a prototype smart parking system based on wireless sensor network technology with features like remote parking monitoring, automated guidance, and parking reservation mechanism is described in [30]. In chapter-9 we provide a case study on a smart parking system.

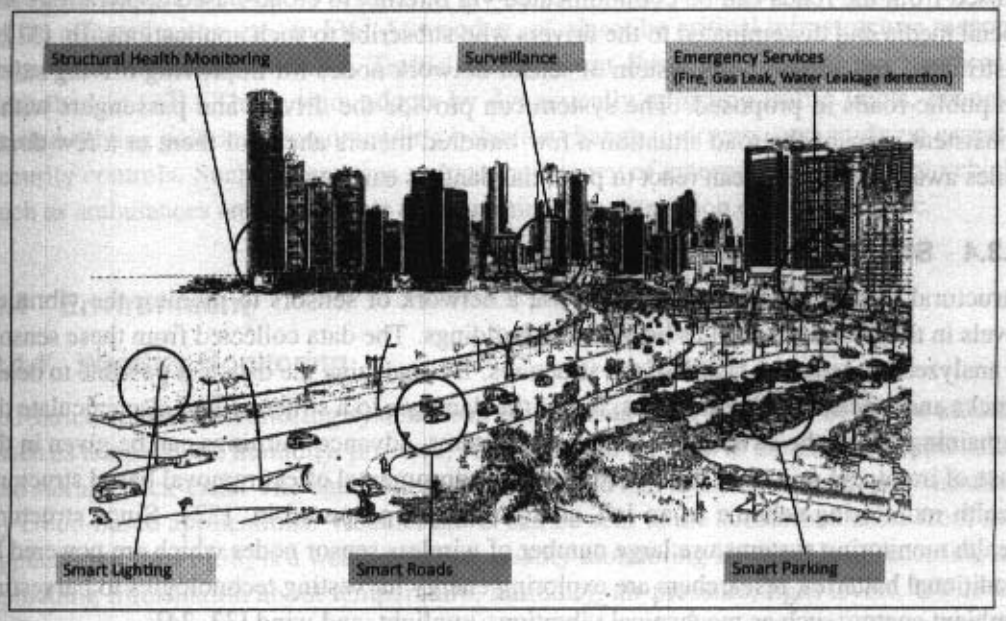


Figure 2.2: Applications of IoT for cities

2.3.2 Smart Lighting

Smart lighting systems for roads, parks and buildings can help in saving energy. According to an IEA report [27], lighting is responsible for 19% of global electricity use and around 6% of global greenhouse gas emissions. Smart lighting allows lighting to be dynamically controlled and also adaptive to the ambient conditions. Smart lights connected to the Internet can be controlled remotely to configure lighting schedules and lighting intensity. Custom lighting configurations can be set for different situations such as a foggy day, a festival, etc. Smart lights equipped with sensors can communicate with other lights and exchange information on the sensed ambient conditions to adapt the lighting. Castro *et. al.* [28] describe the need for smart lighting system in smart cities, smart lighting features and how to develop interoperable smart lighting solutions.

2.3.3 Smart Roads

Smart roads equipped with sensors can provide information on driving conditions, travel time estimates and alerts in case of poor driving conditions, traffic congestions and accidents. Such information can help in making the roads safer and help in reducing traffic jams. Information sensed from the roads can be communicated via Internet to cloud-based applications and social media and disseminated to the drivers who subscribe to such applications. In [31], a distributed and autonomous system of sensor network nodes for improving driving safety on public roads in proposed. The system can provide the drivers and passengers with a consistent view of the road situation a few hundred meters ahead of them or a few dozen miles away, so that they can react to potential dangers early enough.

2.3.4 Structural Health Monitoring

Structural Health Monitoring systems use a network of sensors to monitor the vibration levels in the structures such as bridges and buildings. The data collected from these sensors is analyzed to assess the health of the structures. By analyzing the data it is possible to detect cracks and mechanical breakdowns, locate the damages to a structure and also calculate the remaining life of the structure. Using such systems, advance warnings can be given in the case of imminent failure of the structure. An environmental effect removal based structural health monitoring scheme in an IoT environment is proposed in [32]. Since structural health monitoring systems use large number of wireless sensor nodes which are powered by traditional batteries, researchers are exploring energy harvesting technologies to harvesting ambient energy, such as mechanical vibrations, sunlight, and wind [33, 34].

2.3.5 Surveillance

Surveillance of infrastructure, public transport and events in cities is required to ensure safety and security. City wide surveillance infrastructure comprising of large number of distributed and Internet connected video surveillance cameras can be created. The video feeds from surveillance cameras can be aggregated in cloud-based scalable storage solutions. Cloud-based video analytics applications can be developed to search for patterns or specific events from the video feeds. In [35] a smart city surveillance system is described that leverages benefits of cloud data stores.

2.3.6 Emergency Response

IoT systems can be used for monitoring the critical infrastructure in cities such as buildings, gas and water pipelines, public transport and power substations. IoT systems for fire detection, gas and water leakage detection can help in generating alerts and minimizing

their effects on the critical infrastructure. IoT systems for critical infrastructure monitoring enable aggregation and sharing of information collected from large number of sensors. Using cloud-based architectures, multi-modal information such as sensor data, audio, video feeds can be analyzed in near real-time to detect adverse events. Response to alerts generated by such systems can be in the form of alerts sent to the public, re-routing of traffic, evacuations of the affected areas, etc. In [36] Attwood *et. al.* describe critical infrastructure response framework for smart cities. A Traffic Management System for emergency services is described in [37]. The system adapts by dynamically adjusting traffic lights, changing related driving policies, recommending behavior change to drivers, and applying essential security controls. Such systems can reduce the latency of emergency services for vehicles such as ambulances and police cars while minimizing disruption of regular traffic.

2.4 Environment

2.4.1 Weather Monitoring

IoT-based weather monitoring systems can collect data from a number of sensor attached (such as temperature, humidity, pressure, etc.) and send the data to cloud-based applications and storage back-ends. The data collected in the cloud can then be analyzed and visualized by cloud-based applications. Weather alerts can be sent to the subscribed users from such applications. AirPi [38] is a weather and air quality monitoring kit capable of recording and uploading information about temperature, humidity, air pressure, light levels, UV levels, carbon monoxide, nitrogen dioxide and smoke level to the Internet. In [39], a pervasive weather monitoring system is described that is integrated with buses to measure weather variables like humidity, temperature and air quality during the bus path. In [40], a weather monitoring system based on wireless sensor networks is described. In chapter-9 we provide a case study on a weather monitoring system.

2.4.2 Air Pollution Monitoring

IoT based air pollution monitoring systems can monitor emission of harmful gases (CO_2, CO, NO, NO_2, etc.) by factories and automobiles using gaseous and meteorological sensors. The collected data can be analyzed to make informed decisions on pollutions control approaches. In [41], a real-time air quality monitoring system is presented that comprises of several distributed monitoring stations that communicate via wireless with a back-end server using machine-to-machine communication. In [42], an air pollution system is described that integrates a single-chip microcontroller, several air pollution sensors, GPRS-Modem, and a GPS module. In chapter-9 we provide a case study on an air pollution monitoring system.

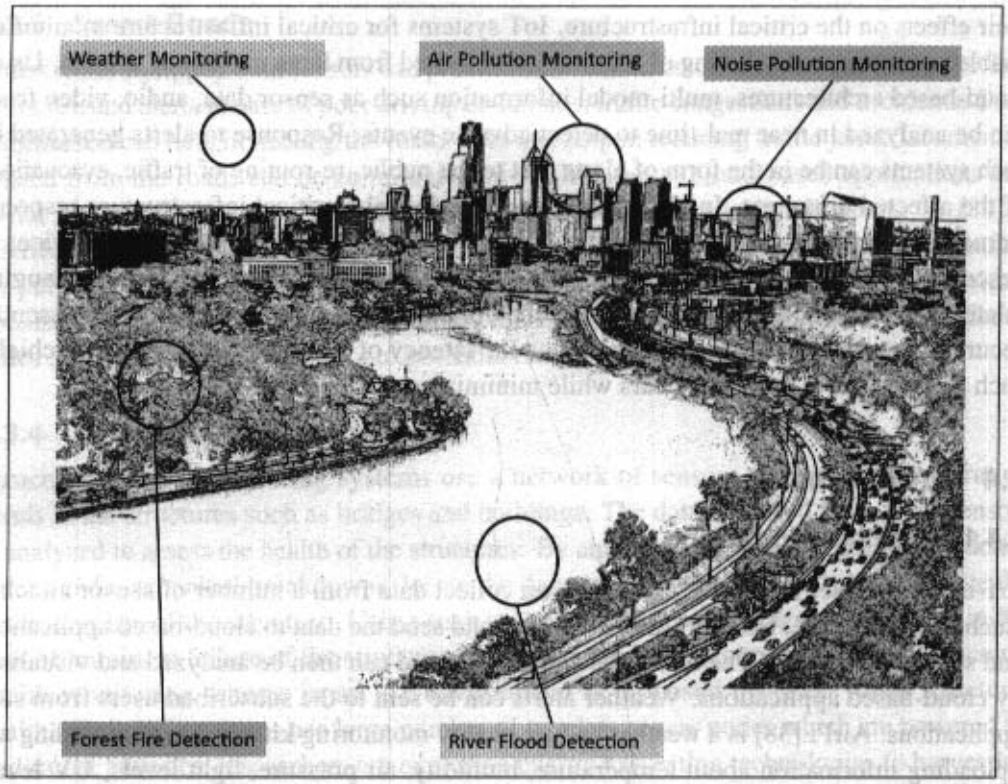


Figure 2.3: Applications of IoT for environment

2.4.3 Noise Pollution Monitoring

Due to growing urban development, noise levels in cities have increased and even become alarmingly high in some cities. Noise pollution can cause health hazards for humans due to sleep disruption and stress. Noise pollution monitoring can help in generating noise maps for cities. Urban noise maps can help the policy makers in urban planning and making policies to control noise levels near residential areas, schools and parks. IoT based noise pollution monitoring systems use a number of noise monitoring stations that are deployed at different places in a city. The data on noise levels from the stations is collected on servers or in the cloud. The collected data is then aggregated to generate noise maps. In [43], a noise mapping study for a city is presented which revealed that the city suffered from serious noise pollution. In [44], the design of smart phone application is described that allows the users to continuously measure noise levels and send to a central server where all generated

information is aggregated and mapped to a meaningful noise visualization map.

2.4.4 Forest Fire Detection

Forest fires can cause damage to natural resources, property and human life. There can be different causes of forest fires including lightening, human negligence, volcanic eruptions and sparks from rock falls. Early detection of forest fires can help in minimizing the damage. IoT based forest fire detection systems use a number of monitoring nodes deployed at different locations in a forest. Each monitoring node collects measurements on ambient conditions including temperature, humidity, light levels, etc. A system for early detection of forest fires is described in [45] that provides early warning of a potential forest fire and estimates the scale and intensity of the fire if it materializes. In [46], a forest fire detection system based on wireless sensor networks in presented. The system uses multi-criteria detection which is implemented by the artificial neural network (ANN). The ANN fuses sensing data corresponding to multiple attributes of a forest fire (such as temperature, humidity, infrared and visible light) to detect forest fires.

2.4.5 River Floods Detection

River floods can cause extensive damage to the natural and human resources and human life. River floods occur due to continuous rainfall which cause the river levels to rise and flow rates to increase rapidly. Early warnings of floods can be given by monitoring the water level and flow rate. IoT based river flood monitoring system use a number of sensor nodes that monitor the water level (using ultrasonic sensors) and flow rate (using the flow velocity sensors). Data from a number of such sensor nodes is aggregated in a server or in the cloud. Monitoring applications raise alerts when rapid increase in water level and flow rate is detected. In [47], a river flood monitoring system in described that measures river and weather conditions through wireless sensor nodes equipped with different sensors. In [48], a motes-based sensor network for river flood monitoring is described. The system includes a water level monitoring module, network video recorder module, and data processing module that provides flood information in the form of raw data, predicted data, and video feed.

2.5 Energy

2.5.1 Smart Grids

Smart Grid is a data communications network integrated with the electrical grid that collects and analyzes data captured in near-real-time about power transmission, distribution, and consumption. Smart Grid technology provides predictive information and recommendations to utilities, their suppliers, and their customers on how best to manage power. Smart

Grids collect data regarding electricity generation (centralized or distributed), consumption (instantaneous or predictive), storage (or conversion of energy into other forms), distribution and equipment health data. Smart grids use high-speed, fully integrated, two-way communication technologies for real-time information and power exchange. By using IoT based sensing and measurement technologies, the health of equipment and the integrity of the grid can be evaluated. Smart meters can capture almost real-time consumption, remotely control the consumption of electricity and remotely switch off supply when required. Power thefts can be prevented using smart metering. By analyzing the data on power generation, transmission and consumption smart girds can improve efficiency throughout the electric system. Storage collection and analysis of smarts grids data in the cloud can help in dynamic optimization of system operations, maintenance, and planning. Cloud-based monitoring of smart grids data can improve energy usage levels via energy feedback to users coupled with real-time pricing information. Real-time demand response and management strategies can be used for lowering peak demand and overall load via appliance control and energy storage mechanisms. Condition monitoring data collected from power generation and transmission systems can help in detecting faults and predicting outages. In [49], application of IoT in smart grid power transmission is described.

2.5.2 Renewable Energy Systems

Due to the variability in the output from renewable energy sources (such as solar and wind), integrating them into the grid can cause grid stability and reliability problems. Variable output produces local voltage swings that can impact power quality. Existing grids were designed to handle power flows from centralized generation sources to the loads through transmission and distribution lines. When distributed renewable energy sources are integrated into the grid, they create power bi-directional power flows for which the grids were not originally designed. IoT based systems integrated with the transformers at the point of interconnection measure the electrical variables and how much power is fed into the grid. To ensure the grid stability, one solution is to simply cut off the overproduction. For wind energy systems, closed-loop controls can be used to regulate the voltage at point of interconnection which coordinate wind turbine outputs and provides reactive power support [52].

2.5.3 Prognostics

Energy systems (smart grids, power plants, wind turbine farms, for instance) have a large number of critical components that must function correctly so that the systems can perform their operations correctly. For example, a wind turbine has a number of critical components, e.g., bearings, turning gears, for instance, that must be monitored carefully as wear and tear in such critical components or sudden change in operating conditions of the machines can

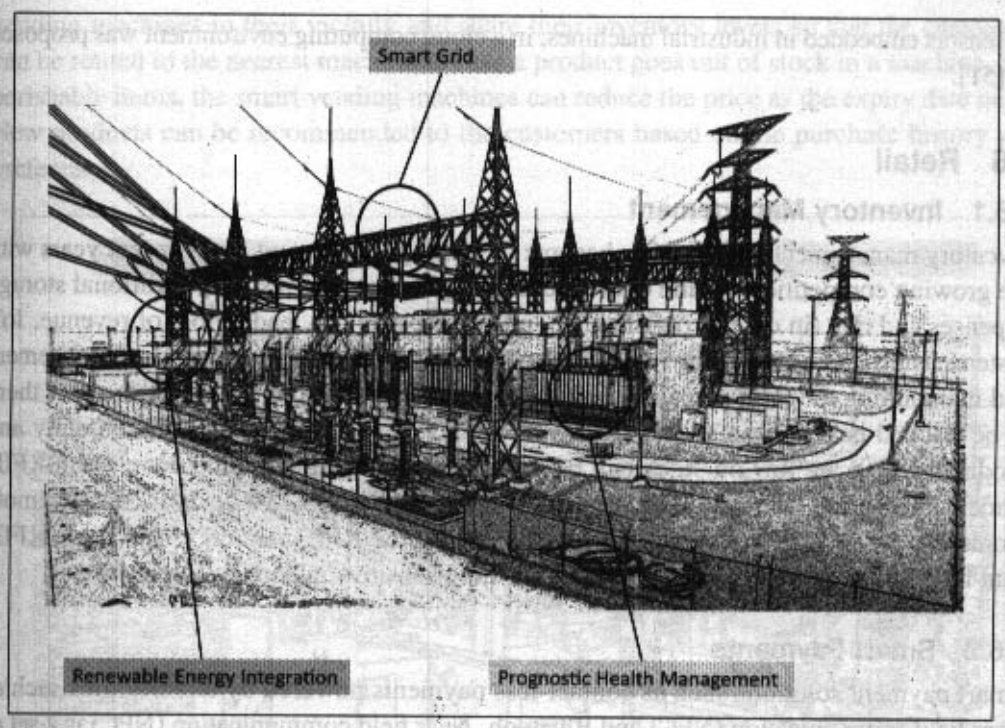


Figure 2.4: Applications of IoT for energy systems

result in failures. In systems such as power grids, real-time information is collected using specialized electrical sensors called Phasor Measurement Units (PMU) at the substations. The information received from PMUs must be monitored in real-time for estimating the state of the system and for predicting failures. Energy systems have thousands of sensors that gather real-time maintenance data continuously for condition monitoring and failure prediction purposes. IoT based prognostic real-time health management systems can predict performance of machines or energy systems by analyzing the extent of deviation of a system from its normal operating profiles. Analyzing massive amounts of maintenance data collected from sensors in energy systems and equipment can provide predictions for the impending failures (potentially in real-time) so that their reliability and availability can be improved. Prognostic health management systems have been developed for different energy systems. OpenPDC [50] is a set of applications for processing of streaming time-series data collected from Phasor Measurement Units (PMUs) in real-time. A generic framework for storage, processing and analysis of massive machine maintenance data, collected from a large number

of sensors embedded in industrial machines, in a cloud computing environment was proposed in [51].

2.6 Retail

2.6.1 Inventory Management

Inventory management for retail has become increasingly important in the recent years with the growing competition. While over-stocking of products can result in additional storage expenses and risk (in case of perishables), under-stocking can lead to loss of revenue. IoT systems using Radio Frequency Identification (RFID) tags can help in inventory management and maintaining the right inventory levels. RFID tags attached to the products allow them to be tracked in real-time so that the inventory levels can be determined accurately and products which are low on stock can be replenished. Tracking can be done using RFID readers attached to the retail store shelves or in the warehouse. IoT systems enable remote monitoring of inventory using the data collected by the RFID readers. In [53], an RFID data-based inventory management system for time-sensitive materials is described.

2.6.2 Smart Payments

Smart payment solutions such as contact-less payments powered by technologies such as Near field communication (NFC) and Bluetooh. Near field communication (NFC) is a set of standards for smart-phones and other devices to communicate with each other by bringing them into proximity or by touching them. Customers can store the credit card information in their NFC-enabled smart-phones and make payments by bringing the smart-phones near the point of sale terminals. NFC maybe used in combination with Bluetooh, where NFC (which offers low speeds) initiates initial pairing of devices to establish a Bluetooh connection while the actual data transfer takes place over Bluetooh. The applications of NFC for contact-less payments are described in [54, 55].

2.6.3 Smart Vending Machines

Smart vending machines connected to the Internet allow remote monitoring of inventory levels, elastic pricing of products, promotions, and contact-less payments using NFC. Smart-phone applications that communicate with smart vending machines allow user preferences to be remembered and learned with time. When a user moves from one vending machine to the other and pairs the smart-phone with the vending machine, a user specific interface is presented. Users can save their preferences and favorite products. Sensors in a smart vending machine monitor its operations and send the data to the cloud which can be used for predictive maintenance. Smart vending machines can communicate with other

vending machines in their vicinity and share their inventory levels so that the customers can be routed to the nearest machine in case a product goes out of stock in a machine. For perishable items, the smart vending machines can reduce the price as the expiry date nears. New products can be recommended to the customers based on the purchase history and preferences.

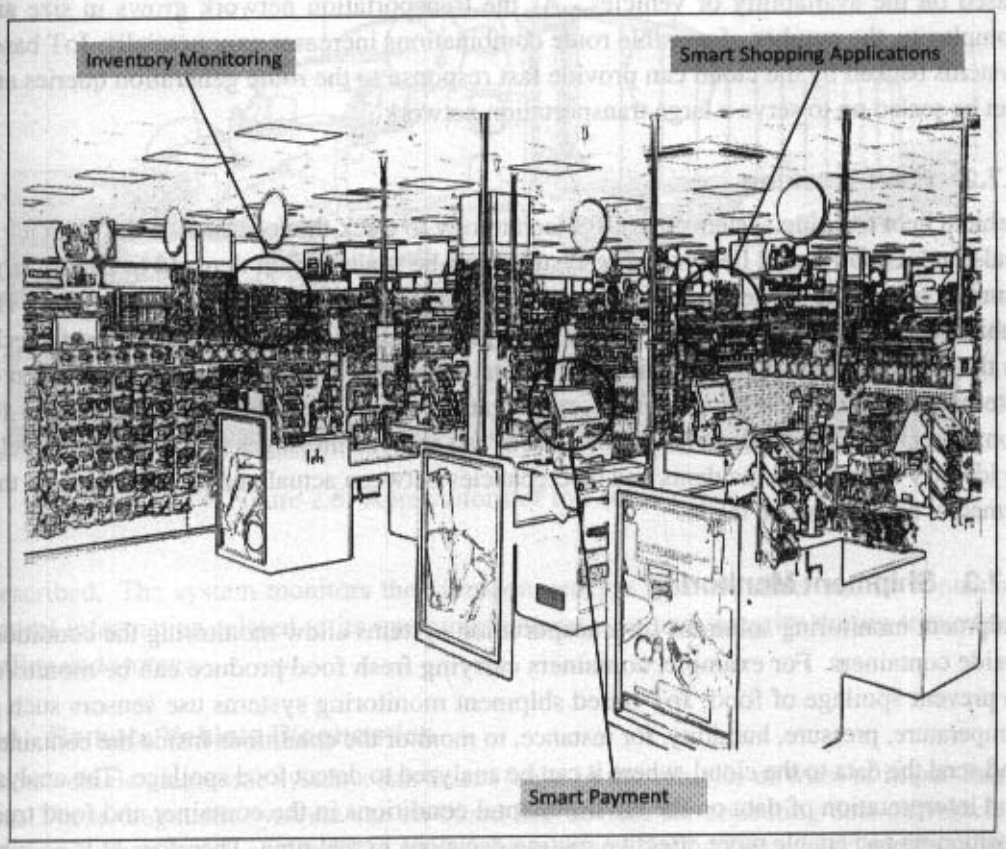


Figure 2.5: Applications of IoT for retail

2.7 Logistics

2.7.1 Route Generation & Scheduling

Modern transportation systems are driven by data collected from multiple sources which is processed to provide new services to the stakeholders. By collecting large amount of

data from various sources and processing the data into useful information, data-driven transportation systems can provide new services such as advanced route guidance [62, 63], dynamic vehicle routing [64], anticipating customer demands for pickup and delivery problem, for instance. Route generation and scheduling systems can generate end-to-end routes using combination of route patterns and transportation modes and feasible schedules based on the availability of vehicles. As the transportation network grows in size and complexity, the number of possible route combinations increases exponentially. IoT based systems backed by the cloud can provide fast response to the route generation queries and can be scaled up to serve a large transportation network.

2.7.2 Fleet Tracking

Vehicle fleet tracking systems use GPS technology to track the locations of the vehicles in real-time. Cloud-based fleet tracking systems can be scaled up on demand to handle large number of vehicles. Alerts can be generated in case of deviations in planned routes. The vehicle locations and routes data can be aggregated and analyzed for detecting bottlenecks in the supply chain such as traffic congestions on routes, assignments and generation of alternative routes, and supply chain optimization. In [58], a fleet tracking system for commercial vehicles is described. The system can analyze messages sent from the vehicles to identify unexpected incidents and discrepancies between actual and planned data, so that remedial actions can be taken.

2.7.3 Shipment Monitoring

Shipment monitoring solutions for transportation systems allow monitoring the conditions inside containers. For example, containers carrying fresh food produce can be monitored to prevent spoilage of food. IoT based shipment monitoring systems use sensors such as temperature, pressure, humidity, for instance, to monitor the conditions inside the containers and send the data to the cloud, where it can be analyzed to detect food spoilage. The analysis and interpretation of data on the environmental conditions in the container and food truck positioning can enable more effective routing decisions in real time. Therefore, it is possible to take remedial measures such as - the food that has a limited time budget before it gets rotten can be re-routed to a closer destinations, alerts can be raised to the driver and the distributor about the transit conditions, such as container temperature exceeding the allowed limit, humidity levels going out of the allowed limit, for instance, and corrective actions can be taken before the food gets damaged. A cloud-based framework for real-time fresh food supply tracking and monitoring was proposed in [61]. For fragile products, vibration levels during shipments can be tracked using accelerometer and gyroscope sensors attached to IoT devices. In [59], a system for monitoring container integrity and operating conditions

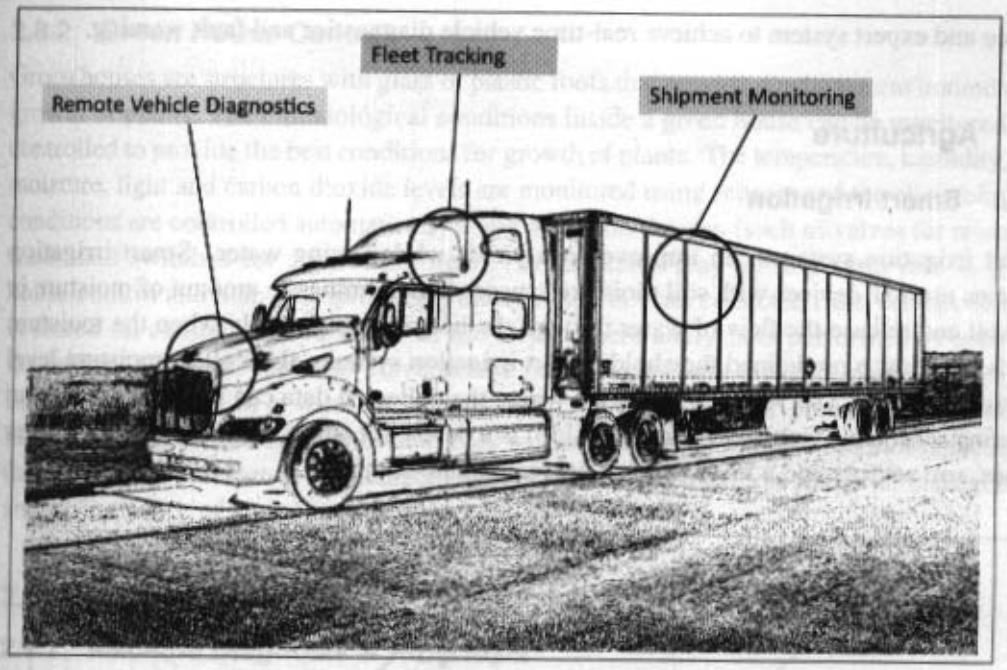


Figure 2.6: Applications of IoT for logistics

is described. The system monitors the vibration patterns of a container and its contents to reveal information related to its operating environment and integrity during transport, handling and storage.

2.7.4 Remote Vehicle Diagnostics

Remote vehicle diagnostic systems can detect faults in the vehicles or warn of impending faults. These diagnostic systems use on-board IoT devices for collecting data on vehicle operation (such as speed, engine RPM, coolent temperature, fault code number) and status of various vehicle sub-systems. Such data can be captured by integrating on-board diagnostic systems with IoT devices using protocols such as CAN bus. Modern commercial vehicles support on-board diagnostic (OBD) standards such as OBD-II. OBD systems provide real-time data on the status of vehicle sub-systems and diagnostic trouble codes which allow rapidly identifying the faults in the vehicle. IoT based vehicle diagnostic systems can send the vehicle data to centralized servers or the cloud where it can be analyzed to generate alerts and suggest remedial actions. In [60], a real-time online vehicle diagnostics and early fault estimation system is described. The system makes use of on-board vehicle diagnostics

device and expert system to achieve real-time vehicle diagnostics and fault warning.

2.8 Agriculture

2.8.1 Smart Irrigation

Smart irrigation systems can improve crop yields while saving water. Smart irrigation systems use IoT devices with soil moisture sensors to determine the amount of moisture in the soil and release the flow of water through the irrigation pipes only when the moisture levels go below a predefined threshold. Smart irrigation systems also collect moisture level measurements on a server or in the cloud where the collected data can be analyzed to plan watering schedules. Cultivar's RainCloud [56] is a device for smart irrigation that uses water valves, soil sensors and a WiFi enabled programmable computer.

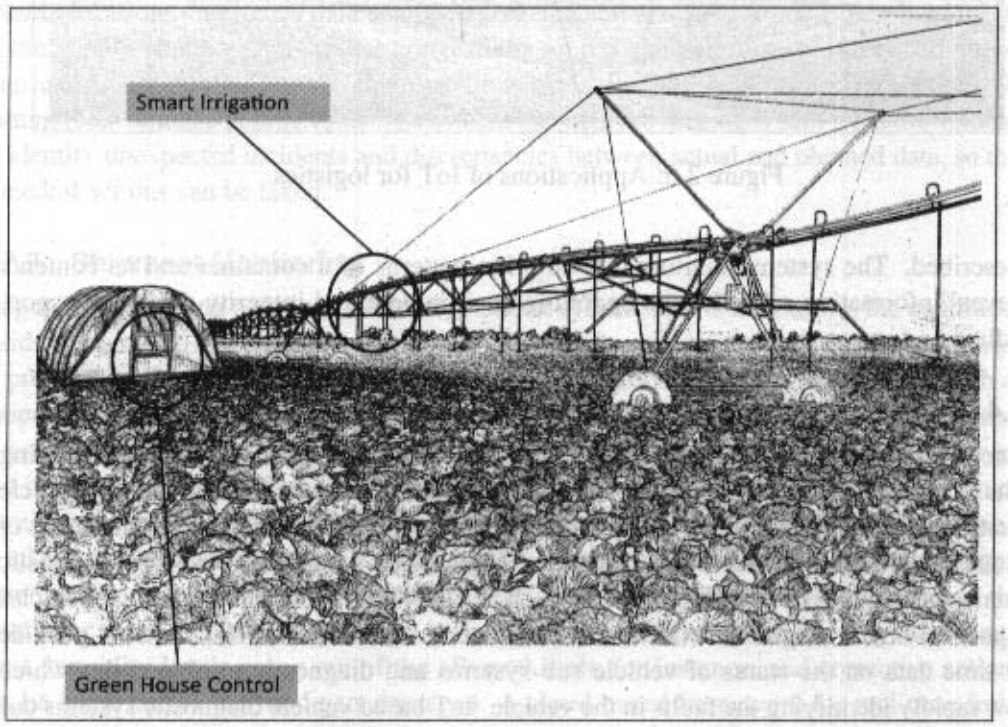


Figure 2.7: Applications of IoT for agriculture

2.8.2 Green House Control

Green houses are structures with glass or plastic roofs that provide conducive environment for growth of plants. The climatological conditions inside a green house can be monitored and controlled to provide the best conditions for growth of plants. The temperature, humidity, soil moisture, light and carbon dioxide levels are monitored using sensors and the climatological conditions are controlled automatically using actuation devices (such as valves for releasing water and switches for controlling fans). IoT systems play an important role in green house control and help in improving productivity. The data collected from various sensors is stored on centralized servers or in the cloud where analysis is performed to optimize the control strategies and also correlate the productivity with different control strategies. In [57], the design of a wireless sensing and control system for precision green house management is described. The system uses wireless sensor network to monitor and control the agricultural parameters like temperature and humidity in real time for better management and maintenance of agricultural production.

2.9 Industry

2.9.1 Machine Diagnosis & Prognosis

Machine prognosis refers to predicting the performance of a machine by analyzing the data on the current operating conditions and how much deviations exist from the normal operating conditions. Machine diagnosis refers to determining the cause of a machine fault. IoT plays a major role in both prognosis and diagnosis of industrial machines. Industrial machines have a large number of components that must function correctly for the machine to perform its operations. Sensors in machines can monitor the operating conditions such as (temperature and vibration levels). The sensor data measurements are done on timescales of few milliseconds to few seconds, which leads to generation of massive amount of data. IoT based systems integrated with cloud-based storage and analytics back-ends can help in storage, collection and analysis of such massive scale machine sensor data. A number of methods have been proposed for reliability analysis and fault prediction in machines. Case-based reasoning (CBR) is a commonly used method that finds solutions to new problems based on past experience. This past experience is organized and represented as cases in a case-base. CBR is an effective technique for problem solving in the fields in which it is hard to establish a quantitative mathematical model, such as machine diagnosis and prognosis. Since for each machine, data from a very large number of sensors is collected, using such high dimensional data for creation of case library reduces the case retrieval efficiency. Therefore, data reduction and feature extraction methods are used to find the representative set of features which have the same classification ability as the complete of

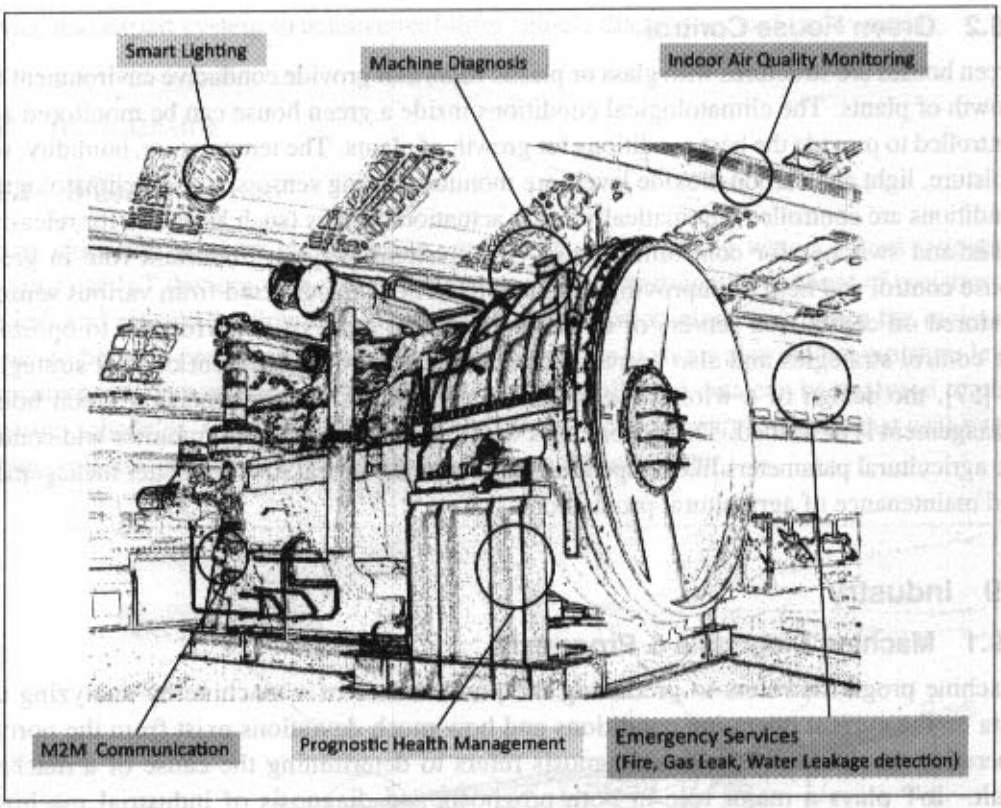


Figure 2.8: Applications of IoT for industry

features. A CBR based machine fault diagnosis and prognosis approach is described in [51]. A survey on recent trends in machine diagnosis and prognosis algorithms is presented in [65].

2.9.2 Indoor Air Quality Monitoring

Monitoring indoor air quality in factories is important for health and safety of the workers. Harmful and toxic gases such as carbon monoxide (CO), nitrogen monoxide (NO), Nitrogen Dioxide (NO_2), etc., can cause serious health problems. IoT based gas monitoring systems can help in monitoring the indoor air quality using various gas sensors. The indoor air quality can vary for different locations. Wireless sensor networks based IoT devices can identify the hazardous zones, so that corrective measures can be taken to ensure proper ventilation. In [66] a hybrid sensor system for indoor air quality monitoring is presented, which contains both stationary sensors (for accurate readings and calibration) and mobile

sensors (for coverage). In [67] a wireless solution for indoor air quality monitoring is described that measures the environmental parameters like temperature, humidity, gaseous pollutants, aerosol and particulate matter to determine the indoor air quality.

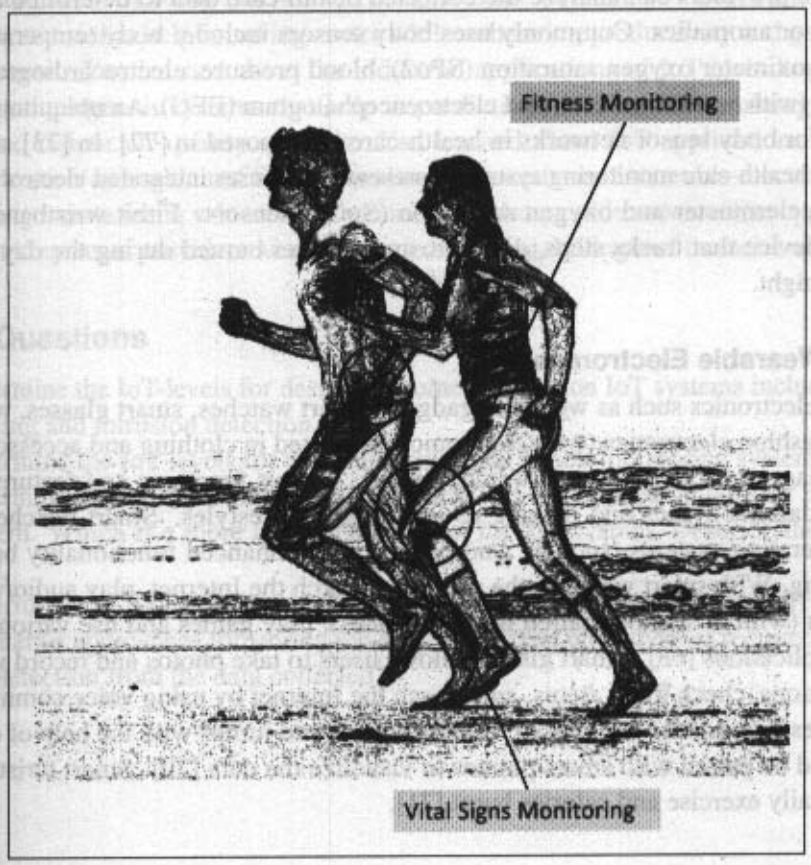


Figure 2.9: Applications of IoT for health

2.10 Health & Lifestyle

2.10.1 Health & Fitness Monitoring

Wearable IoT devices that allow non-invasive and continuous monitoring of physiological parameters can help in continuous health and fitness monitoring. These wearable devices may can be in various forms such as belts and wrist-bands. The wearable devices form a